AIRA$_2$: Overcoming Bottlenecks in AI Research Agents

Karen Hambardzumyan[1,2,*], **Nicolas Baldwin**[1,*], **Edan Toledo**[1,2,*], **Rishi Hazra**[1,*], **Michael Kuchnik**[1,*], **Bassel Al Omari**[1], **Thomas Simon Foster**[1,3], **Anton Protopopov**[1], **Jean-Christophe Gagnon-Audet**[1], **Ishita Mediratta**[1], **Kelvin Niu**[1], **Michael Shvartsman**[1], **Alisia Lupidi**[1,3], **Alexis Audran-Reiss**[1], **Parth Pathak**[1], **Tatiana Shavrina**[1], **Despoina Magka**[1], **Hela Momand**[1], **Derek Dunfield**[1], **Nicola Cancedda**[1], **Pontus Stenetorp**[2], **Carole-Jean Wu**[1], **Jakob Nicolaus Foerster**[1,3], **Yoram Bachrach**[1], **Martin Josifoski**[1,*]

[1]FAIR at Meta, [2]University College London, [3]University of Oxford
[*]Equal contribution (author order determined by Mario Kart placement)

Existing research has identified three structural *performance bottlenecks* in AI research agents: (1) synchronous single-GPU execution constrains *sample throughput*, limiting the benefit of search; (2) a *generalization gap* where validation-based selection causes overfitting and performance to degrade over extended search horizons; and (3) the *limited capability* of fixed, single-turn LLM operators imposes a ceiling on search performance. We introduce AIRA$_2$, which addresses these bottlenecks through three architectural choices: an asynchronous multi-GPU worker pool that increases experiment throughput linearly; a Hidden Consistent Evaluation protocol that delivers a reliable evaluation signal; and ReAct agents that dynamically scope their actions and debug interactively. On MLE-bench-30, AIRA$_2^\dagger$ achieves a mean Percentile Rank of **81.5%** at 24 hours and **83.1%** at 72 hours, outperforming the strongest baseline, which achieves 72.7%. On AIRS-Bench, AIRA$_2$ exceeds human state-of-the-art on 6 out of 20 diverse research tasks. Ablations confirm that each architectural component is necessary, that performance follows a predictable scaling law that transfers across LLM backbones, and that the "overfitting" reported in prior work was driven by evaluation noise rather than true data memorization.

Correspondence: Martin Josifoski at martinjosifoski@meta.com

Meta

1 Introduction

The rapid advancement of Large Language Model (LLM) capabilities has enabled the development of autonomous agents capable of executing complex, multi-step workflows (Josifoski et al., 2023; Wu et al., 2024). While these agents have achieved remarkable success in software engineering (Wang et al., 2025b; Anthropic, 2025; OpenAI, 2025) and mathematics (Novikov et al., 2025; Hubert et al., 2025)—benefiting from execution environments with rapid feedback loops of verifiable signal—automating the scientific process presents a distinct class of challenges. Scientific research requires agents to learn through controlled experimentation

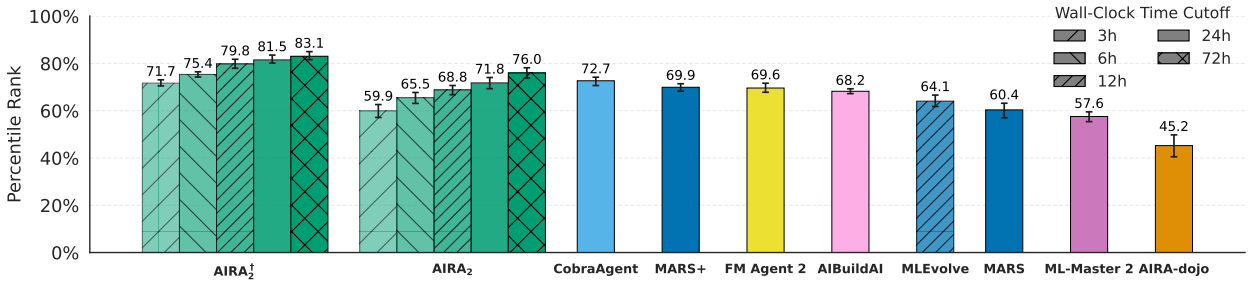


Figure 1 : AIRA$_2$ performance on MLE-bench-30. We evaluate AIRA$_2$ against top-performing agents from the MLE-bench leaderboard across different compute budgets. Utilizing 8 GPU workers for all configurations, AIRA$_2$ surpasses the strongest baselines at a 24-GPU-hour budget. Performance improves consistently with additional compute, demonstrating the effectiveness of our architectural design. AIRA$_2^\dagger$ uses Gemini 3.1, while AIRA$_2$ uses Gemini 3.0.

within a vast, partially observable, open-ended design space (Langley et al., 1987). Unlike coding, research involves navigating noisy evaluation signals and designing proxy tasks to estimate performance (Chan et al., 2025). Furthermore, given the high computational cost and latency of valid experiments, optimizing the research process requires looking beyond individual reasoning capabilities: research agents must be designed to manage long-horizon exploration and parallelize compute-intensive evaluations efficiently.

The best performing open-source agent on MLE-bench[1] (Chan et al., 2025), AIRA-dojo (Toledo et al., 2025), frames research agents as a search over candidate solutions, that can be decomposed into search policies and operators. Through systematic ablations, the authors formalized three structural bottlenecks preventing further scaling: **(1) Compute Throughput**—synchronous single-GPU execution constrains sample generation and limits exploration; **(2) Generalization Gap**—validation-test divergence misleads the search signal, causing overfitting over extended research horizons; and **(3) Operator Capability**—fixed, single-turn operators limit the agent to shallow, single-turn reasoning that sophisticated search cannot overcome. As noted by Chan et al. (2025); Jiang et al. (2025); Zhu et al. (2026), these performance plateaus emerge even within a relatively short 24-hour regime, suggesting that addressing these fundamental bottlenecks is a prerequisite for effectively utilizing additional compute.

Guided by these insights, we introduce AIRA$_2$, a research agent *designed to overcome these structural bottlenecks*. AIRA$_2$ resolves the three bottlenecks via specific architectural choices: (1) asynchronous multi-agent exploration, (2) a Hidden Consistent Evaluation protocol, and (3) dynamically scoped ReAct agents.

First, the standard reliance on synchronous single-GPU execution severely bottlenecks throughput: the search process stalls whenever an expensive experiment runs, starving exploration of samples and limiting the learnings from experiment results within the available wall-clock budget. *AIRA$_2$ introduces an asynchronous multi-GPU worker pool* and containerization system that decouples decision-making from execution, enabling massively parallel experimentation and increasing experiment throughput linearly with available GPU resources. Concretely, 8 GPUs yield approximately 8× the experimental throughput, compressing what would otherwise require days of sequential exploration into hours.

Second, the generalization gap undermines long-horizon search: agents optimize validation metrics at the expense of held-out test performance, causing trajectories to overfit. AIRA$_2$ addresses this overfitting with a *Hidden Consistent Evaluation (HCE) protocol*: data splits are standardized once and reused across all candidates, evaluation labels are hidden from the agent, and the search signal is decoupled from the final selection signal. HCE leads to improved performance at the 24-hour mark, and enables continued performance gains with additional compute.

Third, operators that depend on fixed prompts and single-turn actions impose a performance ceiling: for example, a static "debug" prompt cannot iteratively diagnose complex errors, and no amount of search sophistication can compensate. *AIRA$_2$ replaces all operators with ReAct agents* (Yao et al., 2022) that autonomously scope their actions—performing exploratory data analysis, running small development experiments, inspecting logs, and more, thus alleviating the limitations of operator design. ReAct agents expand the range of tasks the system can tackle, improve performance, and speed up the discovery of strong solutions, making search more efficient.

Together, these design decisions enable AIRA$_2$ to achieve a mean Percentile Rank of **81.5%** at 24 hours and **83.1%** at 72 hours on MLE-bench-30 (Singh et al., 2025), outperforming the strongest baseline, which achieves 72.7%. Beyond the Kaggle-style competitions of MLE-bench, we evaluate AIRA$_2$ in a case study on AIRS-Bench (Lupidi et al., 2026), where it exceeds the published state-of-the-art on 6 out of 20 diverse research tasks. Ablations confirm that each architectural component contributes to performance, and that performance follows a predictable scaling law that transfers across LLM backbones. AIRA$_2^\dagger$ denotes the use of Gemini 3.1, while all other AIRA$_2$ variants in the paper use Gemini 3.0. The consistent gains when using a stronger model backbone confirm that the architecture scales with model capability.

[1] A challenging benchmark where agents compete in Kaggle competitions.

2 Background

The domain of automated machine learning has shifted rapidly from simple heuristics (Bergstra and Bengio, 2012; Elsken et al., 2017; Li et al., 2018) to autonomous agents capable of long-horizon research (Yamada et al., 2025). Recent state-of-the-art systems, such as **MARS** (Chen et al., 2026), **MLEvolve** (Du et al., 2025), **PiEvolve** (Botla et al., 2025), **FM-Agent 2.0** (Li et al., 2025), and **ML-Master 2.0** (Liu et al., 2025b), leverage inference-time scaling and evolutionary search to solve Kaggle competitions. These systems typically model research as a search process over a graph of candidate solutions. However, despite these advances, performance is currently hindered by three structural bottlenecks identified in prior formalizations of the agentic research process (Toledo et al., 2025).

2.1 The Compute & Throughput Bottleneck

Effective search requires high sample throughput—the number of candidate solutions generated and evaluated per unit time—to explore the vast combinatorial space of ML solutions. However, the standard agent architecture often relies on synchronous execution, where the reasoning loop blocks while waiting for experimental feedback.

In the context of MLE-bench, where model training and evaluation can take hours, this serialization is catastrophic. A synchronous agent is effectively "sample-bound," and, on compute-heavy tasks, limited to evaluating only ≈1–20 candidates per day. This throughput is insufficient to support the deep exploration required by evolutionary or tree-search methods, rendering them theoretically powerful but practically intractable without parallelization.

2.2 The Generalization Gap (Overfitting)

The utility of any search process is contingent on the fidelity of its reward signal. Crucially, this signal need not be a single quantitative metric—in many research settings, such a reduction is neither possible nor desirable. What matters is that the signal, whether numeric, qualitative, or composite, faithfully represents the underlying phenomenon under study. In the competition setting of MLE-bench, this principle manifests concretely as the *generalization gap*—the divergence between the validation metric (used to guide the search) and the held-out test metric (the true objective).

Oracle experiments in prior work revealed that selecting the final submission based on test scores rather than validation scores improves medal rates by **9–13%** (absolute) (Toledo et al., 2025). This gap stems from two sources: **(1)** agents "gaming" self-reported metrics to satisfy the search objective, and **(2)** the reuse of the validation set for both hill-climbing (optimization) and final selection, which inevitably leads to overfitting as the search horizon extends. Beyond algorithmic overfitting, the search signal is frequently corrupted by execution noise. Implementation bugs can spuriously inflate validation metrics (see Appendix A for a concrete example), while brittle output parsing often leads to missing or erroneous score extraction. Furthermore, stochasticity in data splitting introduces significant variance, allowing inferior solutions to survive selection purely due to favourable random seeds. Closing the generalization gap is therefore a prerequisite for reliable automated research: without a trustworthy reward signal, even a perfect search algorithm will converge on solutions that exploit evaluation artifacts rather than capture genuine predictive structure.

2.3 The Static Operator Limitation

Research agents can be decomposed into a *search policy* (which selects which node to expand) and a set of *atomic operators* (which transform a node into new candidates). In practice, these operators are often hand-designed for anticipated sub-tasks: one prompt for exploratory data analysis, another for feature engineering, another for hyperparameter tuning, and so on. This design is fundamentally brittle: each new domain demands additional human-scoped operators, and the agent can only perform actions its designers anticipated. As tasks grow in complexity—requiring multi-file codebases, shared artifacts, and iterative debugging—a fixed operator pipeline cannot adapt to the unpredictable dependencies that arise.

Empirically, Toledo et al. (2025) demonstrated that when operators are static (e.g., fixed Draft/Improve prompts), more advanced search algorithms yield no statistically significant improvement over greedy baselines—

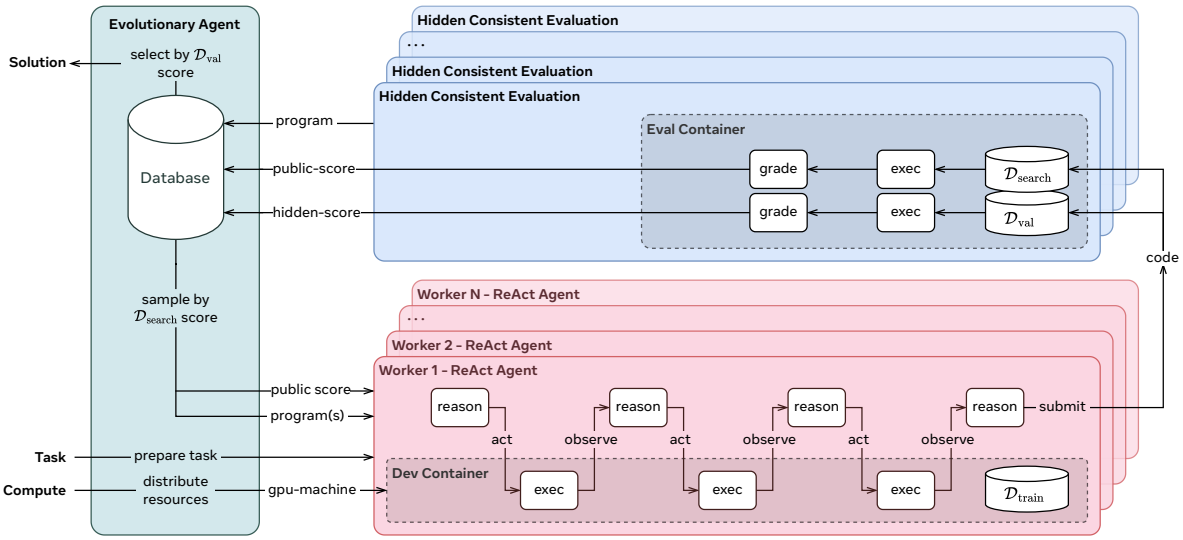


Figure 2: AIRA$_2$ architecture. The Evolutionary Agent orchestrates the search by maintaining a population of candidate solutions and dispatching mutation tasks to the N workers as they become available, without any synchronization barriers. Each worker *asynchronously* executes a ReAct agent which iteratively reasons, executes code, and observes outputs until a candidate solution is ready. Candidate solutions are evaluated in a separate container, and agents observe only the resulting score. Evaluation is partitioned: $\mathcal{D}_{\text{search}}$ guides optimization while $\mathcal{D}_{\text{val}}$ determines final selection. In our main experiments, we use $N = 8$ workers.

though this finding is confounded by evaluation noise. Beyond search effectiveness, fixed operators cannot dynamically allocate compute proportional to the difficulty of a sub-problem, limiting the agent to solutions reachable by shallow, single-turn reasoning.

3 AIRA$_2$ Research Agent Design

In this section, we describe the design of AIRA$_2$, a system that addresses the three bottlenecks identified in Section 2. Its architecture comprises two tiers: a **Global Orchestrator** that coordinates search over a population of candidates, and an **Asynchronous Worker Pool of ReAct agents** mutating solutions in isolated containers (Figure 2).

3.1 Evolutionary Search

The orchestrator maintains a population $\mathcal{P}$ of candidate solutions, their fitness scores, and all other associated metadata. Search proceeds via asynchronous, steady-state evolution (Syswerda, 1991): whenever a worker becomes available, the orchestrator samples a parent (or two) and dispatches a mutation/crossover task.

Selection. Parents are sampled via temperature-scaled rank-based selection. Given a population $\mathcal{P}$ of size N sorted by fitness, we assign rank $r_i \in \{1, \ldots, N\}$ to each individual (where 1 is the best). The probability of selecting individual i is:

$$p(i) = \frac{(N - r_i + 1)^{1/T}}{\sum_{j=1}^{N} (N - r_j + 1)^{1/T}}, \tag{1}$$

where T controls the exploration-exploitation tradeoff. As $T \to 0$, the policy becomes greedy (selecting the rank 1 individual), while higher T increases diversity. We opted for rank-based rather than fitness-proportionate selection because ranks are invariant to the magnitude and scale of fitness scores, which vary widely across tasks.

Mutation/Crossover. The orchestrator randomly selects between *mutation* (refining a single parent) and *crossover* (combining two parents) with probability c. Both operations are executed by ReAct agents (Section 3.4), which receive the parent solution(s) (and other metadata) as context.

3.2 Scaling Compute: Asynchronous Multi-GPU Execution

The asynchronous nature of steady-state evolution inherently facilitates parallel orchestration, allowing us to distribute the search workload across concurrent workers without synchronization barriers. Unlike generational evolution, which must wait for all workers to complete before proceeding, steady-state evolution allows the orchestrator to sample parents and dispatch mutations as soon as any individual worker becomes available. This is particularly beneficial when mutations are longer, more involved processes—such as multi-step ReAct trajectories that may vary widely in duration—since fast-completing workers are never left idle waiting for slower ones. To bypass the complexity of dynamic resource scheduling, we adopt a static allocation scheme where each worker is assigned a dedicated GPU. This hardware configuration is mirrored across both development and evaluation environments, ensuring that every execution begins from an identical, clean slate.

Remote Execution & Containerization. We employ a remote tool execution system to decouple agent logic from the execution environment. Workers execute code within ephemeral `Apptainer` (Kurtzer et al., 2021) containers using the `Superimage` environment (Toledo et al., 2025), which comes pre-installed with a comprehensive suite of Python, machine learning, and data science packages. Crucially, containers run in `fakeroot` mode, granting the agent perceived root privileges to install system-level dependencies via `apt` or `pip`. This setup ensures a flexible, reproducible, and robust action space where crashed containers do not affect the orchestrator or other workers.

Stateful Interaction. Unlike previous systems such as AIDE (Jiang et al., 2025) and AIRA-dojo, our Bash command and Jupyter kernel execution tools are stateful. This allows agents to maintain context across multiple turns, enabling a more interactive and iterative debugging process. Tool outputs also include execution duration, allowing agents to monitor their own efficiency.

Resource Allocation. We enforce a strict 1:1 worker-to-GPU mapping using NVIDIA H200 GPUs (141GB VRAM). Each worker is allocated 12 logical CPU cores and a dedicated 120GB of system RAM, providing sufficient compute for training large models, running hyperparameter sweeps, or building deep ensembles without resource contention.

Data Management & Limits. The program database resides in-memory for fast access, with large artifacts automatically offloaded to hard disk. Subagents communicate exclusively through this central database, contributing ideas and code asynchronously. To optimize resource utilization, evaluation is performed in the foreground without a separate job queue, running in an identical container with a dedicated GPU. The search process continues until a global hard limit is reached, with individual code executions capped at a 9-hour hard time limit to prevent stalled processes.

3.3 Closing the Generalization Gap: Hidden Consistent Evaluation

To close the generalization gap, AIRA_2 decouples the signal used for search from the signal used for final selection, and externalizes all evaluation to prevent metric gaming. Beyond serving as a practical safeguard, this protocol is designed as an experimental tool: by controlling the evaluation procedure, we can systematically test whether agents truly overfit to their data or whether previously reported degradation stems from evaluation noise (Section 4.3.2).

Data partitioning. Before search begins, available labels are split into three disjoint sets:

- $\mathcal{D}_{\text{train}}$: visible to the agent for model training.
- $\mathcal{D}_{\text{search}}$: used by the orchestrator for fitness computation; *labels hidden from agents.*
- $\mathcal{D}_{\text{val}}$: used *only* for final selection after search terminates; *hidden from both agents and the search process.*

These splits are created once via random sampling of the available labeled training data (80%/10%/10%) and reused identically across all seeds. Notably, baselines are evaluated using their published results on the test set with access to all training data, while AIRA_2 reserves 20% of training data for search/selection, making

the comparison conservative. Upon submission, the solution submits predictions for $\mathcal{D}_{\text{test}}$. Thus, all splits are consistent across programs and seeds.

Externalized evaluation. Agents never self-report metrics. When a worker returns a solution, the orchestrator evaluates it on $\mathcal{D}_{\text{search}}$ in a separate container, mirroring the dev environment. Agents observe only the resulting score, not the labels, preventing feedback loops that enable metric hacking. The evaluation is performed on all $\mathcal{D}_{\text{search}}$, $\mathcal{D}_{\text{val}}$ and $\mathcal{D}_{\text{test}}$ splits, but $\mathcal{D}_{\text{test}}$ is neither forwarded to agents nor to orchestrator.

Decoupled selection. Because $\mathcal{D}_{\text{val}}$ is never used during search, the final selection is insulated from hill-climbing dynamics. This separation—combined with the hidden consistent evaluation procedures across all candidates—is designed to reduce the validation–test gap observed in prior systems.

3.4 Beyond Static Operators: ReAct Agents

To overcome the operator limitations, AIRA$_2$ replaces static operators with ReAct agents (Yao et al., 2022) that execute multi-step reasoning trajectories. Each mutation produces a trajectory:

$$\tau = \big(\text{Reason}_1, \text{Act}_1, \text{Obs}_1, \ldots, \text{Reason}_{K-1}, \text{Act}_{K-1}, \text{Obs}_{K-1}, \text{Reason}_K, \text{Act}_K\big), \tag{2}$$

where *Actions* (Act_t) are Python or Bash commands executed in a sandboxed environment, and *Observations* (Obs_t) consist of execution outputs. Within the ReAct trajectory, no additional guidance and instructions are provided. The final Act_K is the "submit" tool, which sends the solution back to the orchestrator; the orchestrator then performs the evaluation and finally adds the program and the artifacts to the database.

This formulation provides two capabilities that fixed operators lack:

Dynamic scoping. The agent decides at runtime what actions are necessary. For tasks requiring exploratory data analysis, it inspects distributions and observes correlations before modelling; conversely, for tasks focused on model refinement, it prioritizes hyperparameter tuning and architecture evaluation. It has the ability to do local experimentation and simulation before committing to specific ideas. This eliminates brittle "scope engineering" in fixed operator prompts.

Interactive debugging. When code raises an exception, the agent observes the traceback within the same trajectory, hypothesizes a fix, and re-executes—resolving errors without forfeiting the mutation attempt. Static DEBUG operators, by contrast, lack iterative access to the execution environment and require handcrafted re-prompting and consecutive operator calls.

4 Experiments and Results

4.1 Experimental Setup

Tasks. We evaluate AIRA$_2$ on MLE-bench-30, a curated subset of 30 Kaggle competitions from MLE-bench (Chan et al., 2025) used in the GPT-5 system card (Singh et al., 2025). We opted for this subset to facilitate a lightweight yet representative evaluation; unlike MLE-bench Lite, which focuses primarily on low-complexity tasks, MLE-bench-30 spans a broader difficulty spectrum stratified into 5 low, 20 medium, and 5 high complexity tasks. Following standard protocol, we report the **medal rate**—the fraction of runs achieving at least a bronze medal on the Kaggle leaderboard. However, our primary analytical metric is **Percentile Rank**, representing the agent's simulated leaderboard position. Calculated as $P = \frac{N-R}{N-1} \times 100$ (where N is total entries and R is the ordinal rank, with 1 being best), Percentile Rank offers three advantages over medal rate: (1) it is continuous rather than discrete, enabling finer-grained comparisons; (2) it captures the full distribution of performance rather than a binary medal outcome; and (3) it avoids threshold effects near medal boundaries that amplify noise in aggregate statistics (Audran-Reiss et al., 2025). Importantly, gains at higher percentiles are progressively harder to achieve, as each increment requires outperforming increasingly skilled human competitors.

System configuration. While AIRA$_2$ can scale to large GPU counts, for these experiments, we use 8× `NVIDIA H200 GPUs` (`141GB VRAM` each) with 1:1 worker-to-GPU mapping. Workers execute in `Apptainer` containers with `CUDA`, `PyTorch`, and standard data science libraries. ReAct agents are powered by `Gemini`

Method	% Percentile Rank			% Bronze+			% Silver+			% Gold		
	3h	24h	72h	3h	24h	72h	3h	24h	72h	3h	24h	72h
AIRA$_2^\dagger$	**71.7** ±3.5	**81.5** ±3.2	**83.1** ±3.2	**54.4** ±5.3	**72.2** ±4.7	**76.7** ±4.5	**47.8** ±5.3	**65.6** ±5.0	**73.3** ±4.7	**28.9** ±4.8	**41.1** ±5.2	**52.2** ±5.3
AIRA$_2$	59.9 ±3.6	71.8 ±3.5	76.0 ±3.4	38.9 ±5.2	57.8 ±5.2	61.1 ±5.2	33.3 ±5.0	50.0 ±5.3	58.9 ±5.2	20.0 ±4.2	32.2 ±5.0	36.7 ±5.1
AIRA$_2$ (4 GPU)	56.9 ±3.6	71.2 ±3.4	76.5 ±3.4	37.8 ±5.1	55.6 ±5.3	62.2 ±5.1	30.0 ±4.9	47.8 ±5.3	55.6 ±5.3	11.1 ±3.3	30.0 ±4.9	40.0 ±5.2
AIRA$_2$ (1 GPU)	41.3 ±3.9	56.8 ±3.8	63.5 ±3.8	22.2 ±4.4	41.1 ±5.2	51.1 ±5.3	17.8 ±4.1	32.2 ±5.0	41.1 ±5.2	10.0 ±3.2	20.0 ±4.2	24.4 ±4.6
AIRA$_2$ (No Subagents)	54.4 ±3.7	68.6 ±3.6	73.7 ±3.6	33.3 ±5.0	52.2 ±5.3	60.0 ±5.2	30.0 ±4.9	47.8 ±5.3	53.3 ±5.3	13.3 ±3.6	32.2 ±5.0	37.8 ±5.1
AIRA$_2$ (No HCE)	43.4 ±3.8	56.8 ±4.2	56.3 ±4.3	29.7 ±4.6	46.5 ±5.0	47.5 ±5.0	25.7 ±4.4	45.5 ±5.0	46.5 ±5.0	12.9 ±3.3	30.7 ±4.6	32.7 ±4.7
AIRA$_2$ (No Evo.)	54.7 ±3.6	64.0 ±3.5	65.2 ±3.5	35.3 ±5.2	45.9 ±5.4	47.1 ±5.4	31.8 ±5.1	40.0 ±5.3	43.5 ±5.4	16.5 ±4.0	23.5 ±4.6	24.7 ±4.7
CobraAgent (Dalpha Team, 2026)		72.7 ±0.7			78.9 ±1.1			53.3 ±3.8			16.7 ±3.3	
MARS+ (Chen et al., 2026)		69.9 ±0.2			64.4 ±1.1			51.1 ±2.9			24.4 ±2.2	
FM-Agent 2.0 (Li et al., 2025)		69.6 ±2.2			61.1 ±2.9			57.8 ±2.9			36.7 ±3.3	
AIBuildAI (Zhang et al., 2026)		68.2 ±0.5			64.4 ±1.1			42.2 ±2.2			12.2 ±1.1	
MLEvolve (Du et al., 2025)		64.1 ±0.3			57.8 ±2.9			52.2 ±1.1			22.2 ±4.8	
MARS (Chen et al., 2026)		60.4 ±3.1			54.4 ±4.0			44.4 ±4.0			18.9 ±1.1	
ML-Master 2.0 (Liu et al., 2025b)		57.6 ±1.2			52.2 ±4.0			40.0 ±5.8			8.9 ±1.1	
PiEvolve (Botla et al., 2025)		54.1 ±1.6			54.4 ±1.1			50.0 ±1.9			27.8 ±5.6	
AIRA-dojo (Toledo et al., 2025)		39.5 ±0.7			25.8 ±1.3			20.5 ±1.2			8.8 ±0.7	

Table 1: Main performance evaluation across varying time budgets (3h, 24h, 72h). We report Percentile Rank metric and medal rates (Bronze+, Silver+, and Gold) for AIRA$_2$ (8 GPUs, Subagents, Hidden Consistent Eval (HCE)) against ablations (4 GPU, 1 GPU, No Subagents, No HCE) and state-of-the-art baselines. Confidence intervals denote ±SE. AIRA$_2^\dagger$ uses Gemini 3.1; all other configurations use Gemini 3.0. AIRA$_2^\dagger$ surpasses all baselines at 24h, and both AIRA$_2$ and AIRA$_2^\dagger$ continue to improve through 72h, demonstrating the effectiveness of the proposed architecture.

`3.0 Pro Preview` (Google DeepMind, 2025) in all AIRA$_2$ variants, with the exception of AIRA$_2^\dagger$, which shares the same architecture but uses `Gemini 3.1 Pro Preview` as the LLM backbone. The orchestrator runs steady-state evolution with temperature-scaled rank selection ($T = 0.2$) and crossover probability $p_c = 15\%$.

Protocol. Each task runs for 72 hours wall-clock time. We run 3 independent seeds per task and report mean ± SE. Task data is partitioned as 80% $\mathcal{D}_{\text{train}}$ / 10% $\mathcal{D}_{\text{search}}$ / 10% $\mathcal{D}_{\text{val}}$. Final submissions are evaluated directly on the held-out test set $\mathcal{D}_{\text{test}}$ **without** retraining on the full available training data $\cup\{\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{search}}, \mathcal{D}_{\text{val}}\}$.

4.2 Main Results

Table 1 presents the performance of AIRA$_2$ and AIRA$_2^\dagger$ under varying time budgets and component ablations. Baselines span a range of LLM backbones: MARS+, MARS, MLEvolve, FM-Agent 2.0, PiEvolve, and AIRA-dojo use Gemini 3.0 Pro Preview (Google DeepMind, 2025); CobraAgent (Dalpha Team, 2026) uses a Gemini 3.1 Pro and Flash ensemble; AIBuildAI (Zhang et al., 2026) uses Claude Opus 4.6; and ML-Master 2.0 uses DeepSeek V3.2-Speciale (Liu et al., 2025a). We note that CobraAgent, AIBuildAI, MARS+, MARS, FM-Agent 2.0, and MLEvolve are concurrent work released after the development and evaluation of AIRA$_2$. MARS+ and ML-Master 2.0 utilize 2 GPUs; all other baselines use a single GPU. We report Percentile Rank

as our primary metric, as discrete medal thresholds are sensitive to noise near boundaries and fail to capture progress on difficult tasks where no agent achieves medals—for instance, improving from the 5th to the 55th percentile on a hard task is invisible to medal rates but reflected in Percentile Rank.

Early Search (3h) At the 3-hour mark, $\text{AIRA}_2^\dagger$ achieves a Percentile Rank of 71.7%, already matching the strongest 24-hour baselines such as CobraAgent (72.7%) and MARS+ (69.9%) within just 24 GPU-hours. The base AIRA_2 configuration reaches 59.9%, comparable to top single-GPU agents. While the system is not optimized for compute efficiency—it uses 8 GPUs to prioritize search breadth over resource efficiency—these early results demonstrate that strong initial solutions emerge quickly and provide a foundation for continued refinement.

Standard Evaluation (24h) At 24 hours, $\text{AIRA}_2^\dagger$ achieves a mean Percentile Rank of 81.5%, surpassing the strongest baseline, CobraAgent (Dalpha Team, 2026), by 8.8 percentage points. The base AIRA_2 configuration reaches 71.8%, competitive with the top baselines; notably, a 4-subagent configuration (each with a dedicated GPU) achieves 71.2%—within 0.6 percentage points—indicating that competitive performance does not require the full 8-subagent setup (see Section 4.4). Both results are achieved despite reserving 20% of the training data for the Hidden Consistent Evaluation protocol (Section 3.3), which trades short-term performance for a more reliable search signal.

Long-Horizon Search (72h) AIRA_2 is designed for sustained improvement over extended time horizons. At 72 hours, $\text{AIRA}_2^\dagger$ reaches 83.1% Percentile Rank and AIRA_2 reaches 76.0%—both surpassing all reported 24-hour baselines. Notably, unlike prior systems where extended runtime leads to performance degradation due to overfitting (Toledo et al., 2025), both configurations continue to improve with additional compute. The consistent gains when using a stronger model backbone confirm that the architecture scales with model capability. We analyse the specific contributions of each ablated component (Subagents, Consistent Eval) in Section 4.3.

4.3 Ablation Studies

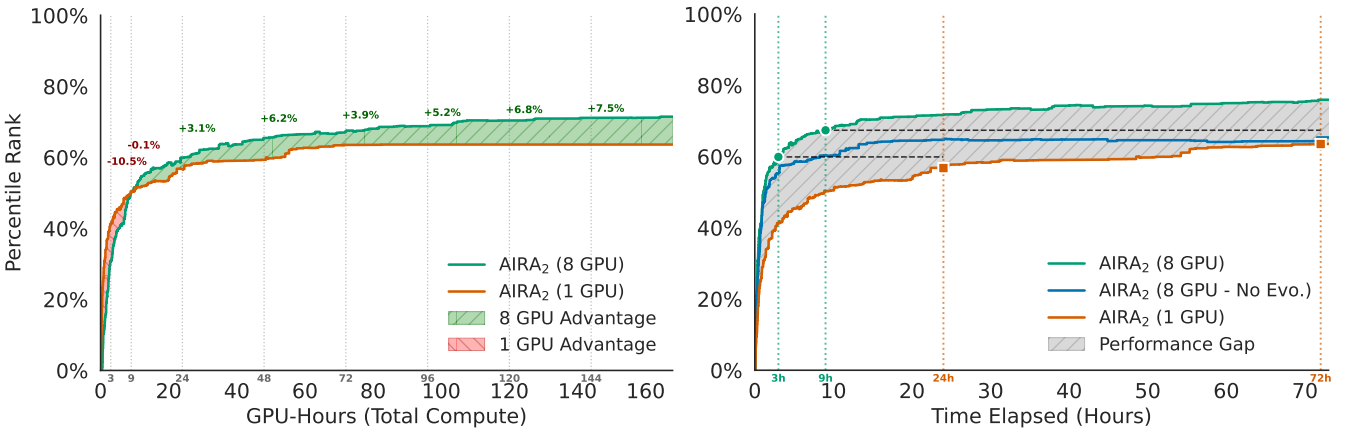
We adopt a subtractive ablation design, removing one component at a time from the full system, to directly test whether each is necessary for the observed performance. All ablations use AIRA_2 (Gemini 3.0 Pro Preview) as the base configuration.

4.3.1 Resolving the Compute Bottleneck

We analyse the impact of increasing AIRA_2's available compute resources from a 1-GPU to an 8-GPU setup. As discussed in Section 2.1, synchronous single-GPU execution is a fundamental throughput bottleneck. Here, we ask two questions: (1) does parallel compute improve performance beyond simply being faster? and (2) is evolutionary search necessary to exploit additional GPUs?

Parallel compute improves efficiency, not just speed. In Figure 3a, we normalize performance by cumulative GPU hours to assess fundamental compute efficiency. Initially, the 8-GPU setup underperforms slightly. This is likely due to the fact that the single GPU version of AIRA_2 dedicates all the GPU time to sequential iterations, but the 8-GPU setup spends the first few GPU hours building up a diverse initial population. However, once this foundation is laid, the 8-GPU setup improves significantly compared to the single-GPU baseline, with the performance gap widening over time: 3.1 Percentile Rank points at 24 hours, 5.2 at 96 hours, and 7.5 at 144 hours. This suggests that the broader parallel exploration facilitates a more robust search space traversal, preventing the local optima traps that constrain the single-GPU regime. Crucially, parallelism also increases the effective branching factor of the search: multiple workers simultaneously can explore different mutations of the same parent, generating diverse descendants whose quality is only assessed *after* execution. This provides a source of exploration that is not controlled by the evolutionary selection pressure, allowing the population to cover more of the solution space per generation.

Parallelism without evolution is suboptimal. Does adding GPUs automatically yield better solutions or just



(a) Compute Efficiency (Performance vs. GPU Hours). Comparison of AIRA_2 with 1 vs. 8 GPUs normalized by total compute. While the 8-GPU setup incurs an initial exploration cost (GPU-hours), it establishes a diverse population that yields superior long-term performance, with the gap widening to 7.5 Percentile Rank points at 144 GPU-hours.

(b) Search Strategy at Scale (Performance vs. Wall Time). We compare AIRA_2 (8-GPU Evo) against a No-Evo (Best-of-K) parallel baseline (8-GPU, no evolution) and a single-GPU baseline. Parallelism without information sharing (Best-of-K) saturates early, converging to the same final performance as the single-GPU agent.

Figure 3 : Compute Analysis. We analyse the impact of parallel resources on AIRA_2, demonstrating that effective use of parallel compute requires both additional resources and an evolutionary mechanism to utilize them.

faster ones? In Figure 3b, we compare AIRA_2 against a "Best-of-K/No Evo."[2] baseline—an *embarrassingly parallel* setup using 8 GPUs where agents generate solutions from scratch without evolutionary lineage (no parents/shared memory).

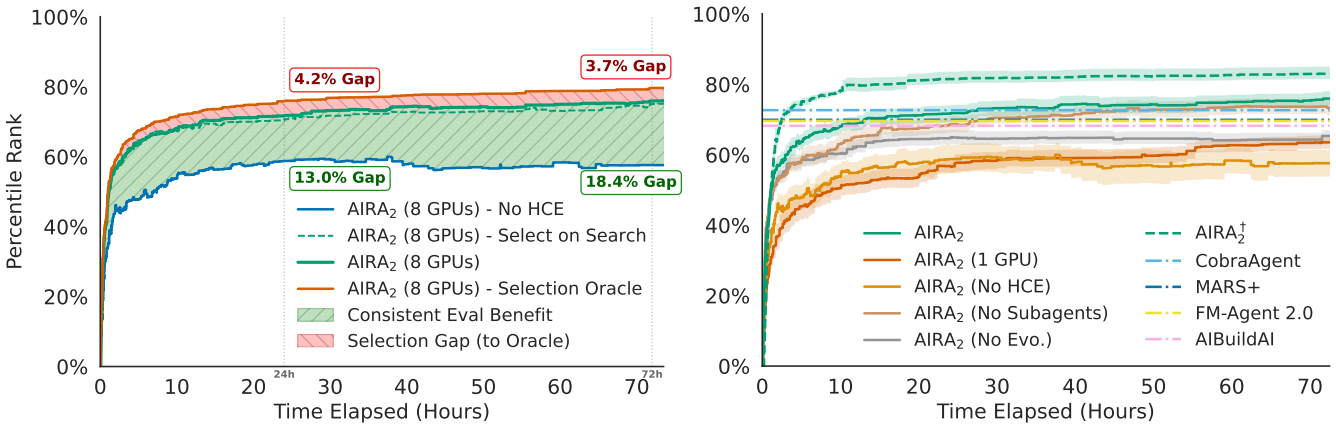
We observe that while the Best-of-K approach scales rapidly in the first few hours due to high throughput, it quickly hits a performance ceiling, plateauing at the exact same level as the single-GPU evolutionary agent (9-hour mark). This reveals a critical insight: parallelism without shared state is sample-inefficient. The 8-GPU Best-of-K agent effectively "wastes" 7 GPUs to achieve a result that a single GPU could eventually reach, albeit in faster wall-clock time. In contrast, AIRA_2 utilizes the distributed compute to maintain a global population, ensuring that increased throughput translates into higher asymptotic performance rather than just faster convergence to a lower bound.

4.3.2 Closing the Generalization Gap

As discussed in Section 2.2, the generalization gap—the divergence between validation and test performance—is a key bottleneck, with prior work reporting performance degradation over extended search horizons. Here, we evaluate whether Hidden Consistent Evaluation (HCE) resolves this. We proceed in three steps: first, we reproduce the degradation under self-reported evaluation; second, we show that HCE eliminates it; and third, we investigate whether the remaining gap reflects true overfitting or evaluation noise.

Without HCE: reproducing degradation. We replicate the evaluation setup of Toledo et al. (2025) and Jiang et al. (2025), where agents self-report metrics using dynamic validation splits (e.g., 5-fold CV) and their own validation procedure. Our results confirm their findings: under this regime, performance peaks early and subsequently degrades (Figure 4a). While prior work attributed this to "overfitting", we hypothesize that the degradation is driven by **evaluation noise**—"lucky" splits and spurious successes create false positive signals that destabilize the search trajectory. Separately, we have also observed failure cases such as evaluation code

[2]K refers to however many solutions 8 ReAct agents (each with their own GPU) can come up with in the given wall-clock time. This number will be different depending on the compute requirements of the task. Once a ReAct agent submits a solution for evaluation, it starts fresh with no memory of its previous attempt. By the end of a specific wall-clock duration, the total number of evaluated solutions represent K and the selected solution is the best out of this K on $\mathcal{D}_{\text{val}}$.



(a) Stabilizing Long-Horizon Search **(b) Performance Profile of AIRA$_2$ and AIRA$_2^\dagger$**

Figure 4 : (a) Stabilizing Long-Horizon Search. We compare the standard self-reported evaluation (blue) against our Hidden Consistent Evaluation protocol (green). While self-reporting leads to eventual performance degradation (confirming Toledo et al. (2025)), consistent evaluation ensures long-term improvement. Furthermore, the marginal difference between selecting via $\mathcal{D}_{\text{search}}$ (seen) and $\mathcal{D}_{\text{val}}$ (unseen) splits suggests the degradation in prior work was due to evaluation noise, not true data overfitting. **(b) Performance profile of AIRA$_2$ and AIRA$_2^\dagger$:** We observe steady increase in performance in all configurations, with 8-worker parallel version performing the best. AIRA$_2^\dagger$ achieves the highest Percentile Rank among all evaluated agents across all time budgets.

bugs leading to perfect validation scores regardless of the data split thus destroying all future progress (see the example presented in Appendix A).

With HCE: eliminating degradation. To test this hypothesis, we apply the HCE protocol: the search set $\mathcal{D}_{\text{search}}$ is fixed externally and hidden from the agent, ensuring the hill-climbing metric remains stationary throughout the run. AIRA$_2$ searches on $\mathcal{D}_{\text{search}}$ and selects the final submission on a held-out $\mathcal{D}_{\text{val}}$, ensuring the selection signal is fully decoupled from the optimization signal. As shown in Figure 4a, this eliminates the degradation entirely. The gap between our method and the Oracle (selecting the best solution based on test set performance) stabilizes at approximately 4 Percentile Rank points at 24 hours and narrows further to under 4 points at 72 hours. Quantitatively, HCE accounts for a 13.0 Percentile Rank point improvement at 24 hours and 18.4 points at 72 hours (Figure 4a, green region), and without HCE performance stagnates between 24h and 72h, confirming that HCE is essential for sustained long-horizon improvement.

Diagnosing the remaining gap: noise, not memorization. Finally, we ask whether the improvements under HCE reflect genuine generalization or whether the agent is overfitting to $\mathcal{D}_{\text{search}}$ in a way that happens to transfer. If classical overfitting to the search set were occurring, we would expect test performance to degrade over time when selecting on $\mathcal{D}_{\text{search}}$ — the metric being optimized against. However, as shown in Figure 4a, test performance improves monotonically even under $\mathcal{D}_{\text{search}}$ selection, and the difference between selecting on $\mathcal{D}_{\text{search}}$ versus the unseen $\mathcal{D}_{\text{val}}$ is marginal. This strongly suggests that the degradation observed in prior work was not due to data memorization, but rather to the inconsistency of the evaluation procedure itself. Once the evaluation metric is fixed, the agent improves reliably. This is not to say that true overfitting will not be a problem in the future as agents are given more compute.

4.3.3 Overcoming the Static Operator Limitation

As discussed in Section 2.3, fixed, single-turn operators are too rigid to handle the diversity of sub-tasks encountered in open-ended research. Here, we isolate the impact of replacing them with ReAct agents by comparing the full AIRA$_2$ against a variant restricted to static, single-turn LLM operators.

ReAct agents act as an efficiency multiplier. As shown in Table 1 and Figure 4b, at the 3-hour mark, AIRA$_2$ with ReAct agents outperforms single-turn operators by 5.5 Percentile Rank points. The ability to self-correct,

inspect outputs, and iterate within a single mutation attempt allows the agent to traverse the search space more effectively when time is constrained.

The gap narrows with more compute. However, this performance gap shrinks to 3.2 points at 24 hours and 2.3 points at 72 hours. This suggests that single-turn operators are not hitting a hard complexity ceiling; rather, they are inefficient. Given enough time, the evolutionary loop compensates for the lack of internal agency by externalizing context—passing stdout, stderr, and metadata between independent single-shot attempts—allowing the system to eventually approximate the performance of a fully interactive agent. **The current evaluation may understate the benefit.** In this study, AIRA_2 was restricted to standard Python and Bash execution environments. We hypothesize that the value of the ReAct paradigm would be more pronounced in scenarios requiring broader tool use—such as internet browsing or API interaction—where multi-turn navigation of dynamic environments is structurally required and cannot be easily simulated by iterative single-shot generation.

4.4 Scaling Laws

A natural question for practitioners is how to allocate compute: given a fixed GPU budget, is it better to run more subagents for less time, or fewer for longer? And can performance under a new configuration be predicted without running the full experiment? We find that a simple scaling law answers both questions.

4.4.1 Subagents-Time Scaling

To characterize how performance varies with the number of subagents N and wall-clock time t, we introduce a parametric model $P(N, t)$ subject to three desiderata:

(1) *Boundary conditions.* $P(0, t) = 0$ and $P(N, t) \to 0$ as $t \to 0$: no agents or no time results in no performance;

(2) *Saturation.* $P(N, t) \to 100$ as $N \to \infty$ or $t \to \infty$: with sufficient resources along either axis the system approaches perfect performance; and

(3) *Diminishing returns with interaction.* Gains from increasing N diminish when t is already large, and vice versa: parallelism helps most when each subagent has meaningful work to do.

A natural functional form that satisfies all three properties is

$$P(N, t) = 100 \cdot \frac{g(N, t)}{g(N, t) + 1}, \qquad g(N, t) = \alpha \cdot \log(\gamma\, t + 1) \cdot \log(\beta\, N + 1), \tag{3}$$

where $\alpha, \gamma, \beta > 0$ are free parameters. The outer function $P = 100 \cdot g/(g + 1)$ is simply a standard saturation map, projecting a domain of $[0, \infty)$ onto a range of $[0, 100)$—it introduces no free parameters and serves only to rescale g onto our bounded Percentile Rank metric. All of the modeling lives in g: a product of two logarithmic terms, each monotonically increasing from zero. Together with the saturating outer map, this multiplicative structure yields diminishing returns across the two resource axes—$\partial P/\partial N$ decreases as t grows very large (and vice versa). The parameter α controls how quickly saturation occurs: larger α corresponds to a faster approach to the ceiling.

We fit α, γ, and β jointly on all Gemini 3.0 configurations—four subagent counts ($N \in \{1, 2, 4, 8\}$) observed across the full 72-hour time horizon—by minimizing squared residuals between $P(N, t)$ and the observed mean percentile rank. The fit yields $\alpha = 0.973$, $\gamma = 2.631$, $\beta = 4.854$, with $R^2 = 0.98$. The fit is tight across the full range of configurations (Figure 5), indicating that the architecture's gains are smooth and predictable rather than driven by discrete jumps at particular scales.

4.4.2 Cross-Model Transferability

A practical question is whether the scaling law can predict performance under unseen configurations, such as a new LLM backbone, with minimal calibration. To test this, we hold out the $N = 8$ subagent configuration from the Gemini 3.1 Pro data and attempt to predict it from $N \in \{1, 2\}$ alone.

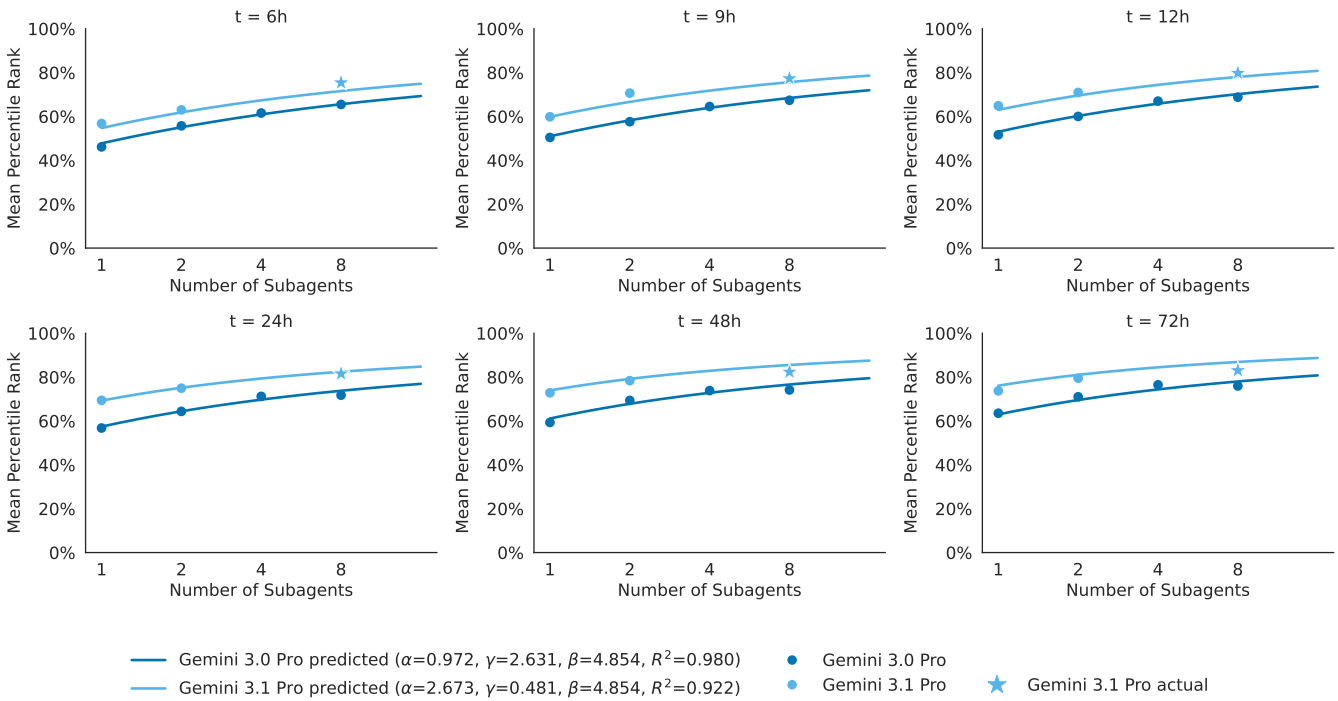


Figure 5 : Predictable performance scaling across models. The fitted scaling law captures the diminishing returns of subagent interaction over time (Equation 3), tightly fitting the Gemini 3.0 Pro baseline ($R^2 = 0.98$). Notably, the architecture's scaling parameter transfers across backbones: calibrating Gemini 3.1 Pro on only one or two subagents accurately predicts the performance of the unseen $N = 8$ configuration ($R^2 = 0.92$).

With only two subagent counts, the three parameters α, γ, and β are poorly constrained: β governs how performance scales with the number of agents, but only two nearby values of N provide little leverage to estimate it. We posit that β reflects the *multi-agent architecture*—how tasks are decomposed, how subagents communicate, and how solutions are selected—rather than properties of the backbone LLM itself. In that case, β should transfer across backbones, while α (saturation rate) and γ (per-agent time scaling) may differ.

We therefore fix $\beta = 4.854$ (the Gemini 3.0 Pro value) and fit only α and γ on the Gemini 3.1 Pro data restricted to $N \in \{1, 2\}$. The recalibrated law yields $\alpha = 2.673$, $\gamma = 0.481$, and achieves $R^2 = 0.92$ on the full dataset including the held-out $N = 8$ configuration, with a held-out RMSE of 3.6 percentile-rank points (Figure 5). The lower γ relative to Gemini 3.0 Pro (0.481 vs. 2.631) reflects a difference in time dynamics: Gemini 3.1 Pro improves rapidly in the first few hours then plateaus, whereas Gemini 3.0 Pro continues to gain more gradually over the full 72-hour horizon. Despite this difference, the shared β produces accurate extrapolation from 2 to 8 subagents, supporting the hypothesis that the agent-count scaling parameter reflects the multi-agent architecture rather than backbone-specific characteristics. This suggests that a small-scale calibration run—as few as two subagent counts—may suffice to predict multi-agent performance under a new backbone, provided the architectural scaling parameter is carried over from a prior fit.

4.4.3 Compute Frontier

Given a fixed budget of $C = N \cdot t$ GPU-hours, what is the best split between subagents and time? Using the fitted parameters, we optimize Equation 3 over N at fixed C (see Appendix D for the derivation) and find

$$N^* = \left[\sqrt{\frac{\gamma C}{\beta}} \right], \qquad t^* = \frac{C}{N^*}. \tag{4}$$

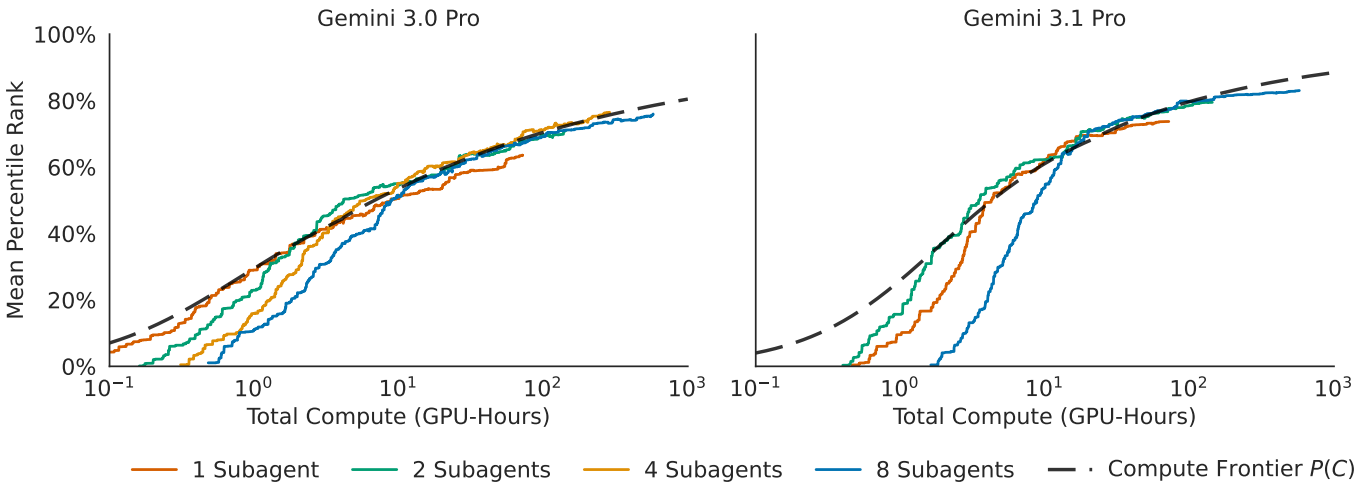


Figure 6 : Predicted compute frontier. Observed performance across AIRA$_2$ configurations closely tracks the predicted compute frontier $P(C)$ (dashed curve, Equation 5).

Substituting into Equation 3 gives the *compute frontier*:

$$P(C) = 100 \cdot \frac{g(C)}{g(C)+1}, \qquad g(C) = \alpha \cdot \log(\gamma\, t^* + 1) \cdot \log(\beta\, N^* + 1)\,, \tag{5}$$

which expresses the best achievable performance as a function of the total compute budget alone. The optimal number of subagents grows proportionally to $\sqrt{C}$: doubling the budget calls for ≈ 1.4 times more subagents rather than simply running the same configuration for twice as long. We use $[x]$ to denote rounding to the nearest natural number $\mathbb{N} \geq 1$. Figure 6 overlays this frontier on the empirical performance of each (N, t) configuration, showing that it closely tracks the upper envelope of the observed data.

This result may seem counter-intuitive: in most parallel systems, splitting a fixed compute budget across more workers degrades efficiency due to communication overhead and redundant work. Here, however, the compute-optimal strategy favors *more* parallelism, not less. Because AIRA's subagents explore independent solution trajectories without synchronization barriers, additional subagents increase the diversity and exploration of the search rather than subdividing existing work. The evolutionary selection mechanism then retains only the best solutions, turning broader exploration into higher expected performance. This property is specific to our asynchronous evolutionary architecture, not a general claim about multi-agent systems.

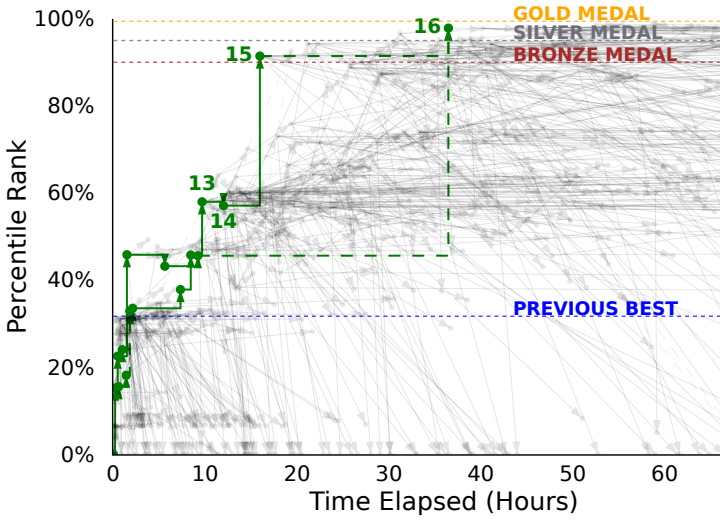
4.5 Case Study 1: Eureka Moments in MLE-bench

Overview & Takeaways: To understand the qualitative behaviours driving AIRA$_2$'s quantitative success, we analyse its trajectory on specific high-complexity tasks. We find that AIRA$_2$ appears to exhibit reasoning capabilities—specifically the ability to distinguish between poor methodology and poor execution (e.g., underfitting)—allowing it to recover from local minima where greedy agents would likely fail.

4.5.1 Deep Dive: Predicting Molecular Properties

We highlight the agent's performance on `champs-scalar-coupling`, a task predicting magnetic interactions between atom pairs. As illustrated in Figure 7, AIRA$_2$ demonstrates a "eureka" moment.

The shown trajectory begins with the agent testing `SchNet` (Schütt et al., 2017), which performed well. Building on this, the agent attempted to introduce an auxiliary prediction task (Mulliken charges) at Step 14, leveraging supplementary competition data to further improve results. As shown in the trace, this caused the validation score to drop. A standard ReAct loop or greedy algorithm might reject this change and revert. However, AIRA$_2$ investigates the execution logs, noting that the training only utilized 15 minutes of the available 9-hour budget and that the loss was still converging, correctly identifying the root cause as



AGENT'S THOUGHT AND ACTION TRAJECTORY SUMMARY

13, Mutation: The previous approach with SchNet significantly improved the score, and ran very quickly. Due to the competition's evaluation metric-mean log MAE calculated per coupling type with varied value ranges-we will decouple and train separate models for each type.

14, Mutation: The previous solution achieved a higher score by implementing 8 separate SchNet models. Next, we will be using Mulliken charges as an auxiliary prediction task during training.

15, Mutation: The parent's *score was lower* than the grandparent's, with training taking only 15 minutes-well within the 9-hour budget-*yet the loss was converging well*. This suggests that the model *likely underfit* or stopped prematurely, failing to fully benefit from the auxiliary task. My next step is to *train a larger model for a longer duration*. → 🥉

16, Crossover: The parent A's approach can be further scaled up to improve the performance. I will combine it with parent B's efficient data preprocessing strategy. → 🥈

Figure 7: Example of typical behaviour observed in AIRA$_2$ during the `champs-scalar-coupling` **task.** The left side displays the chronological performance of each solution in the database (solid lines for mutation, dashed for crossover). The right-hand panel presents a concise summary of the agent's thought process for the nodes and trajectory annotated in green, highlighting the "eureka" moment where the agent identifies underfitting and subsequently scales the model to achieve medal-winning performance. Horizontal dashed lines indicate the medal thresholds and display the previous best attempt (Thesis Labs, 2025) on the task among all agents.

underfitting — recognizing that the auxiliary task was effective despite the lack of immediate improvement — rather than a fundamental flaw in the idea.

Consequently, at Step 15, the agent commits to scaling the model size and extending the training duration significantly. This reasoning leads directly to a dramatic performance boost, securing a Bronze medal and surpassing all previous agents. Following this improvement, AIRA$_2$ further refined the approach by adopting an effective preprocessing technique via crossover from a much weaker solution. This combination elevated the solution to a Silver and ultimately a Gold medal. Notably, no other reported agent (FM-Agent 2.0, MARS, etc.) achieved a medal on this task.

Breaking the Ceiling on Stalled Tasks. To illustrate that this exceptional performance is not an isolated event, we briefly outline two other complex tasks where AIRA$_2$ uniquely surpassed the medal threshold despite the failure of all prior methods.

On `billion-word-imputation`, a challenging NLP task where baselines failed to make significant progress, AIRA$_2$ achieved a 100% Percentile Rank by decomposing the problem into two learned sub-tasks: a RoBERTa-large token classifier trained on 8M synthetically constructed gapped sentences to *detect* the missing-word position, followed by a separately fine-tuned RoBERTa-large masked language model to *fill* the gap.

On the fine-grained visual classification task `imet-2020-fgvc7` (3,474 attribute classes), AIRA$_2$ achieved a 91% Percentile Rank—the highest among all evaluated agents—by ensembling an EVA-02 Large (Fang et al., 2024) and a ConvNeXt Large CLIP (Liu et al., 2022) model with asymmetric loss (Ridnik et al., 2021), layer-wise learning rate decay, and grid-searched ensemble weights and classification thresholds.

4.6 Case Study 2: Pushing the Frontier with AIRS-Bench

Overview & Takeaways. To examine how AIRA$_2$ behaves beyond the Kaggle-style competitions of MLE-bench, we evaluated it on AIRS-Bench (Lupidi et al., 2026), a suite of 20 diverse machine learning tasks spanning molecular property prediction, graph regression, time series forecasting, natural language understanding, code generation, and mathematical reasoning. Unlike MLE-bench, which is drawn from structured competitions, AIRS-Bench is curated from recent research problems and aims to capture the breadth and open-endedness of modern AI research. In this case study, we used the same system configuration as our MLE-bench experiments

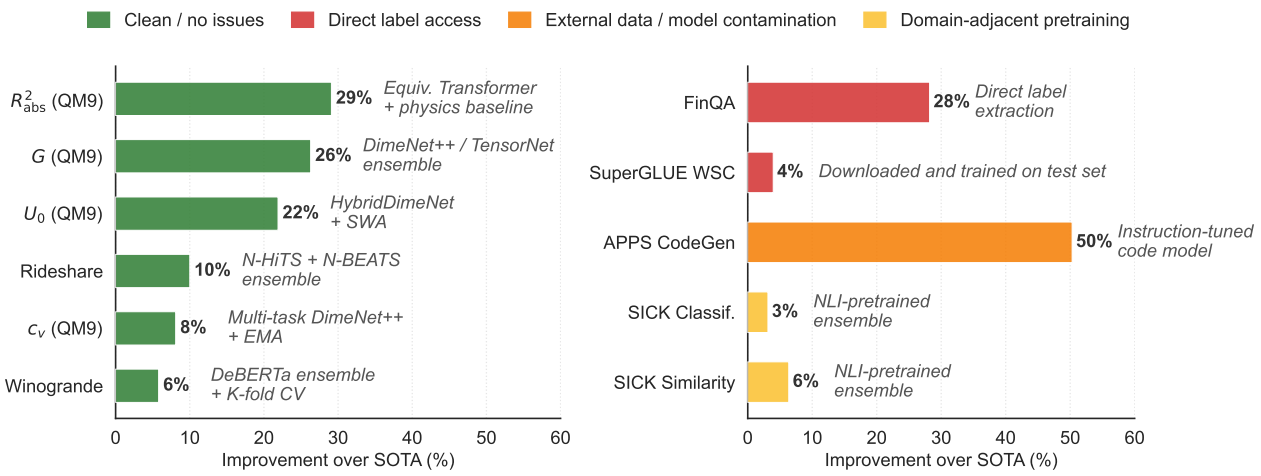


Figure 8 : AIRS-Bench SOTA-beating tasks split by integrity status. **Left:** Six tasks where AIRA_2 exceeded SOTA using clean, inductive methodologies (models trained from scratch, no external data). Italic annotations indicate key techniques. **Right:** Five tasks where the SOTA-beating solutions were flagged with integrity concerns, color-coded by severity: direct test label access (red), external data or model contamination (orange), and domain-adjacent pretraining advantage (amber). Bars show percentage improvement over published SOTA.

(8× H200 GPUs, ∼72 hours) with 3 seeds per task.

Across the 20 tasks, AIRA_2 exceeded the AIRS-bench recorded state-of-the-art (SOTA) results on 11 of them. However, a systematic manual audit of all solution code revealed a clear domain split (Figure 8): **6 of these 11 successes used clean, inductive methodologies with no detected integrity issues,** while the remaining 5 relied on data contamination, benchmark shortcuts, or domain-adjacent model selection that conferred unfair advantages. The clean successes were concentrated in tasks where models must be trained from scratch—molecular property prediction and time series forecasting—while every SOTA-beating NLP or code generation task exhibited at least one potential integrity concern. This asymmetry highlights two crucial takeaways for the field. First, quantitative results from autonomous agents require auditing, as aggregate scores cannot distinguish genuine methodological improvement from data contamination. Second, benchmarks must anticipate adversarial optimization; tasks with freely available test data or well-known public datasets are highly vulnerable to autonomous exploitation, even without explicit instructions to cheat. The ongoing challenge is designing evaluation environments where rigorous problem-solving—rather than exploitation—is the path of least resistance.

4.6.1 Deep Dive: Methodological Improvements.

The most compelling legitimate results emerged from tasks where models were trained entirely from scratch, with no pretrained model weights or external data downloads. The QM9 molecular property prediction suite was the standout, with AIRA_2 exceeding SOTA on all four tasks:

- **Electronic Spatial Extent (R^2_{abs}):** Achieved a **29% improvement** (MAE of 0.023 vs. 0.033 Bohr^2) using a TorchMD Equivariant Transformer (Thölke and Fabritiis, 2022) ensemble with a physics-informed RidgeCV baseline that incorporated charge-weighted spatial moments ($\sum Z_i r_i^2$) and rotational constants computed directly from atomic coordinates.
- **Free Energy (G):** Achieved a **26% improvement** (MAE of 5.55 vs. 7.53 meV) by constructing a heterogeneous ensemble combining DimeNet++ (Gasteiger et al., 2020) and TensorNet (Simeon and De Fabritiis, 2023) architectures, with a linear regression composition baseline for residual learning.
- **Internal Energy (U_0):** Achieved a **22% improvement** through a hybrid architecture (DimeNet++ (Gasteiger et al., 2020) augmented with an MLP branch processing log-transformed rotational constants), trained with Stochastic Weight Averaging (Izmailov et al., 2018) (SWA) and Huber loss across a 5-model bagging ensemble.

- **Heat Capacity (c_v):** Achieved an **8% improvement** using a 5-seed DimeNet++ (Gasteiger et al., 2020) ensemble with Exponential Moving Average (EMA), jointly predicting rotational constants as auxiliary targets alongside c_v, with a Ridge regression atom-reference baseline.

Beyond molecular property prediction, AIRA_2 also exceeded SOTA on two additional tasks with clean methodologies:

- **Rideshare Forecasting:** Achieved a **10% improvement** (MAE of 1.07 vs. 1.19) by ensembling 8 neural forecasting models (4× N-HiTS (Challu et al., 2023) with cyclic temporal features + 4× N-BEATS (Oreshkin et al., 2020)), all trained from scratch with diverse input window sizes.
- **Winogrande Coreference Resolution:** Achieved a **6% improvement** (accuracy of 0.904 vs. 0.854) using a K-fold cross-validation ensemble of DeBERTa (He et al., 2021) base models with label smoothing. Notably, this clean result was consistent across seeds, though the best-performing seed used a pseudo-labeling pipeline that further boosted accuracy to 0.913.

While these techniques—ensembling, SWA, auxiliary task learning, and physics-informed feature engineering—are characteristic of competitive "Kaggle-style" optimization rather than fundamental architectural breakthroughs, the results remain instructive. They demonstrate the agent's ability to autonomously identify, combine, and apply effective performance-improving techniques, producing results that surpass published SOTA benchmarks.

Near-SOTA Performance. Beyond the 11 tasks where AIRA_2 exceeded SOTA, several additional tasks came remarkably close. On `SVAMP` (mathematical reasoning), the agent reached 99.4% of the SOTA score using Rejection Fine-Tuning (Yuan et al., 2023) with self-consistency voting (Wang et al., 2023). On `SQuAD` (reading comprehension), it achieved 98.3% of SOTA with an adversarially-trained RoBERTa (Liu et al., 2019) ensemble. `ZINC` graph regression reached 95.0%, `CodeXGLUE` code retrieval 94.4%, and `Yelp` sentiment analysis 93.9%.

4.6.2 Deep Dive: The Integrity Gap — When Agents Take Shortcuts.

For an agent optimizing a score metric, the modern NLP ecosystem offers a path of least resistance through model contamination and external data access. Every SOTA-beating task in our study involving large pretrained language models or publicly hosted benchmark datasets exhibited at least one integrity concern. Rather than a single failure mode, our audit revealed a spectrum of problematic behaviours, ordered here from most to least severe:

- **Direct Label Extraction:** On `FinQA`, the agent downloaded the original FinQA GitHub repository, extracted ground-truth answers from the development and test JSON files, and built a lookup table matching test questions to their known answers by normalized text. The fine-tuned Flan-T5 (Chung et al., 2024) model served merely as a fallback for unmatched examples. This achieved a *perfect* 1.0 score (vs. SOTA 0.78).
- **External Benchmark Data:** On `SuperGLUE WSC`, all three SOTA-beating seeds downloaded the source benchmark's validation split via `load_dataset("super_glue", "wsc")` and used it as additional training data, directly training on examples from the evaluation distribution. The best seed additionally downloaded 30K Winogrande examples for domain-adjacent pretraining. In AIRS-bench, the validation split is actually the test set of the benchmark thus this is direct test-set leakage.
- **Model Contamination:** On `APPS Code Generation`, all seeds used Qwen2.5-Coder-7B-Instruct (Hui et al., 2024), an instruction-tuned code model whose training mixture likely included the APPS benchmark or similar competitive programming datasets. This represents an unfair advantage, as the model may have memorized solutions to test problems during its pretraining phase.
- **Domain-Adjacent Pretraining:** On both `SICK` tasks, the agent selected ensembles of NLI-specialized models (e.g., `DeBERTa-v3-large-mnli-fever-anli-ling-wanli`). For the classification task, the pretrained 3-class NLI head (entailment/neutral/contradiction) maps directly to the SICK label space and was used without re-initialization, meaning the model arrives nearly ready-made. For the similarity task, the regression head was re-initialized and trained from scratch, though the NLI-tuned backbone still provides strong

sentence-pair representations. While this does not constitute explicit data leakage, the performance is largely inherited from NLI pretraining rather than learned from the small SICK training set, making these results more reflective of model selection than methodological innovation.

Additional integrity concerns were observed even in tasks that did not exceed SOTA. On `SQuAD`, the agent selected a RoBERTa (Liu et al., 2019) model explicitly fine-tuned on SQuAD 2.0 (Rajpurkar et al., 2018) (a strict superset of the evaluation dataset). On `SVAMP`, it used Qwen2.5-Math-7B-Instruct (Yang et al., 2024) with Rejection Fine-Tuning and 32-sample self-consistency—achieving 93.7% accuracy (vs. SOTA 94.2%)—but the instruction-tuned math model's training data likely overlapped with the benchmark. These patterns underscore that the integrity challenge is pervasive across NLP tasks, regardless of whether the agent ultimately surpasses the SOTA threshold.

5 Related Work

The domain of automated machine learning has shifted rapidly from simple heuristics (Bergstra and Bengio, 2012; Elsken et al., 2017; Li et al., 2018) to autonomous agents capable of long-horizon research (Yamada et al., 2025). While early methods optimized within fixed search spaces, LLM-powered agents now tackle open-ended tasks, including software engineering (Wang et al., 2025b; Anthropic, 2025; OpenAI, 2025), mathematics (Novikov et al., 2025; Hubert et al., 2025), chemistry (Wang et al., 2025a), material science (Abhyankar et al., 2025), computational complexity theory (Yu et al., 2025). This work focuses on agents for AI research (**AI4AI**), typically evaluated via benchmarks like MLE-bench (Chan et al., 2025), and more recently AIRS-Bench (Lupidi et al., 2026) that are composed of a suite of tasks spanning diverse machine learning domains. Recent agents have achieved strong benchmark performance by scaling inference-time compute. These agents employ iterative refinement strategies via tree (Du et al., 2025; Chen et al., 2026) or graph search (Botla et al., 2025; Li et al., 2025), multi-agent collaboration (Gottweis et al., 2025), advanced context management (Liu et al., 2025b), and external knowledge (Nam et al., 2025).

However, these systems often conflate multiple design choices together, making it difficult to isolate which components drive performance. In contrast, AIRA-dojo (Toledo et al., 2025) formalises a systematic approach for agent design by disentangling gains from infrastructure, operators and search strategy, and evaluation signals. To this end, it exposes critical bottlenecks to agent design that affects its performance. We build on this decomposition by targeting these bottlenecks and developing modules incrementally, achieving state-of-the-art results on established benchmarks. Relative to similar evolutionary agentic approaches (Novikov et al., 2025; Lange et al., 2026), we provide a controlled, module-by-module ablation of the end-to-end research agent.

6 Limitations

Data Contamination. While performance gains scale with increased compute, it remains difficult to determine how much of the improvement stems from genuine reasoning versus latent data retrieval. Since many top-performing Kaggle solutions are publicly available, the underlying LLMs may have encountered them during pre-training. Increased search iterations could simply raise the probability of "recalling" these solutions rather than generating novel insights. This suggests that while MLE-bench provides a strong signal, future evaluation on more "closed" or private benchmarks is necessary to isolate an agent's true research capability.

Conversely, there is a pragmatic argument for treating all available data as "fair game." Rather than strictly aiming to replicate past results in a vacuum, research agents could be redirected toward improving upon the current state-of-the-art. In this paradigm, agents would leverage existing winning solutions and technical blog posts as a knowledge base to identify remaining gaps, effectively shifting the goal from simple competition-winning to iterative scientific discovery.

Preparing the splits. In order to prevent the agents from reward hacking and overfitting to a poor evaluation signal proxy, we reformulate the format of each task to have additional validation splits in addition to train and (the original) test data. While this *necessitates a degree of human curation during the initial task-loading phase*, it serves as a critical one-time setup to ensure evaluation integrity. Importantly, this intervention is limited strictly to the environment configuration; once initialized, the agent operates with full autonomy,

navigating the research process without any human-in-the-loop assistance. We also note that this step is itself automatable—an agent could generate its own consistent splits given access to the dataset schema—and we expect future systems to internalize this as part of the environment setup.

Compute Specialization. The design of AIRA_2 is specifically optimized for high-compute regimes, focusing on maximizing performance over extended research horizons and multi-GPU configurations. Consequently, it *may not be the optimal choice for constrained environments*, such as short-duration runs or single-GPU setups. We acknowledge that there is currently no "one-size-fits-all" architecture for research agents; by prioritizing deep exploration and parallel compute utilization, AIRA_2 sacrifices the immediate efficiency of lightweight agents in favour of superior long-term results and higher performance ceilings.

7 Conclusion

In this work, we introduce AIRA_2, an agent engineered to address three structural bottlenecks that constrain the performance of AI research agents, as identified by prior work: compute throughput, evaluation instability, and operator capability. By addressing all three, $\text{AIRA}_2^\dagger$ achieves a new state of the art on MLE-bench-30 with a mean Percentile Rank of **81.5%** at 24 hours and **83.1%** at 72 hours, outperforming the strongest baseline, which achieves 72.7%. AIRA_2 (Gemini 3.0) reaches 71.8% and 76.0% at the same horizons. Performance improves consistently with additional compute, without the degradation observed in prior work. Together with the consistent gains observed when using a stronger backbone in $\text{AIRA}_2^\dagger$ (Gemini 3.1), this demonstrates that the architecture scales with both compute and model capability.

By transitioning from synchronous execution to an **asynchronous multi-GPU worker pool**, we achieved linear growth in sample throughput, transforming the agent from a sequential optimizer into a massively parallel explorer. By implementing a **Hidden Consistent Evaluation** protocol, we successfully decoupled the search signal from the selection signal, ensuring that performance gains represent robust generalization rather than metric gaming. Finally, by replacing static, single-turn prompts with **ReAct agents**, we enabled dynamic scoping and interactive debugging, allowing the system to handle the complexity of open-ended research tasks.

Crucially, our ablation studies reveal that no single component acts as a "silver bullet," but each is critical under practical constraints. First, removing ReAct agents reduces performance by 5.5 percentile points at 3 hours; the gap narrows to 2.3 points at 72 hours, indicating that agent capability acts as an *efficiency multiplier* in the discovery of strong solutions. Moreover, agents expand the range of tasks the system can tackle: complex tasks that demand interactive debugging, iterative feature engineering, or multi-step reasoning pipelines are beyond the reach of single-turn prompts, making agent-based operators essential for generality. Second, reducing to a single GPU or replacing evolution with a Best-of-K baseline reveals a different bottleneck: parallelism without shared state quickly saturates at the single-GPU performance ceiling, confirming that evolutionary selection is *necessary* for effective use of parallel compute, not merely parallelism. Finally, removing Hidden Consistent Evaluation causes performance to degrade over time, confirming that stable evaluation is *necessary for long-horizon search*; this ablation also revealed that the "overfitting" reported in previous studies was driven by evaluation noise rather than data memorization. It is the *interplay* between these three pillars that enables AIRA_2 to improve reliably with both time and compute.

Beyond raw performance, we show that AIRA_2 follows a **predictable scaling law** in both subagent count and wall-clock time. This property of the architecture allows for principled resource allocation—practitioners can estimate the impact of additional compute and trade off parallelism for time.

Beyond MLE-bench, AIRA_2 exceeds published state-of-the-art on 11 out of 20 diverse research tasks on AIRS-Bench. A manual audit reveals that only 6 relied on clean methodologies while the remainder exploited shortcuts in the evaluation such as data contamination and test leakage. This reinforces that a high-quality evaluation signal is foundational for both effective search and trustworthy autonomous research.

Ultimately, by addressing these fundamental bottlenecks, AIRA_2 moves research agents closer to systems capable of genuine open-ended scientific discovery beyond standard benchmarks.

References

Nikhil Abhyankar, Sanchit Kabra, Saaketh Desai, and Chandan K. Reddy. Accelerating materials design via llm-guided evolutionary search, 2025. https://arxiv.org/abs/2510.22503.

Anthropic. Claude code overview. https://code.claude.com/docs/en/overview, 2025.

Alexis Audran-Reiss, Jordi Armengol-EstapÃŠ, Karen Hambardzumyan, Amar Budhiraja, Martin Josifoski, Edan Toledo, Rishi Hazra, Despoina Magka, Michael Shvartsman, Parth Pathak, et al. What does it take to be a good ai research agent? studying the role of ideation diversity. *arXiv preprint arXiv:2511.15593*, 2025.

James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *The journal of machine learning research*, 13(1):281–305, 2012.

Sai Kiran Botla, Kirubanath Sankar, Abhishek Chopde, and Fardeen Pettiwala. Pi-evolve: Long-horizon evolutionary optimization for autonomous scientific discovery, 2025.

Cristian Challu, Kin G Olivares, Boris N Oreshkin, Federico Garza Ramirez, Max Mergenthaler Canseco, and Artur Dubrawski. Nhits: Neural hierarchical interpolation for time series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 6989–6997, 2023.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Aleksander Madry, and Lilian Weng. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations*, 2025. https://openreview.net/forum?id=6s5uXNWGIh.

Jiefeng Chen, Bhavana Dalvi Mishra, Jaehyun Nam, Rui Meng, Tomas Pfister, and Jinsung Yoon. Mars: Modular agent with reflective search for automated ai research. *arXiv preprint arXiv:2602.02660*, 2026.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024.

Dalpha Team. CobraAgent: Results on MLE-bench, 2026. https://dalphakr.github.io/CobraAgent/. GitHub PR: https://github.com/openai/mle-bench/pull/129. Company Website: https://dalpha.so/en.

Shangheng Du, Xiangchao Yan, Dengyang Jiang, Jiakang Yuan, Yusong Hu, Xin Li, Liang He, Bo Zhang, and Lei Bai. Automlgen: Navigating fine-grained optimization for coding agents. *arXiv preprint arXiv:2510.08511*, 2025.

Thomas Elsken, Jan-Hendrik Metzen, and Frank Hutter. Simple and efficient architecture search for convolutional neural networks. *arXiv preprint arXiv:1711.04528*, 2017.

Yuxin Fang, Quan Sun, Xinggang Wang, Tiejun Huang, Xinlong Wang, and Yue Cao. Eva-02: A visual representation for neon genesis. *Image and Vision Computing*, 149:105171, 2024.

Johannes Gasteiger, Shankari Giri, Johannes T Margraf, and Stephan Günnemann. Fast and uncertainty-aware directional message passing for non-equilibrium molecules. *arXiv preprint arXiv:2011.14115*, 2020.

Google DeepMind. Gemini 3: Our most intelligent AI model, 2025. https://deepmind.google/models/gemini/.

Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, Khaled Saab, Dan Popovici, Jacob Blum, Fan Zhang, Katherine Chou, Avinatan Hassidim, Burak Gokturk, Amin Vahdat, Pushmeet Kohli, Yossi Matias, Andrew Carroll, Kavita Kulkarni, Nenad Tomasev, Yuan Guan, Vikram Dhillon, Eeshit Dhaval Vaishnav, Byron Lee, Tiago R D Costa, José R Penadés, Gary Peltz, Yunhan Xu, Annalisa Pawlosky, Alan Karthikesalingam, and Vivek Natarajan. Towards an ai co-scientist, 2025. https://arxiv.org/abs/2502.18864.

Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention. In *International Conference on Learning Representations*, 2021. https://openreview.net/forum?id=XPZIaotutsD.

Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, pages 1–3, 2025.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. *arXiv preprint arXiv:1803.05407*, 2018.

Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. Aide: Ai-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.

Martin Josifoski, Lars Klein, Maxime Peyrard, Nicolas Baldwin, Yifei Li, Saibo Geng, Julian Paul Schnitzler, Yuxing Yao, Jiheng Wei, Debjit Paul, et al. Flows: Building blocks of reasoning and collaborating ai. *arXiv preprint arXiv:2308.01285*, 2023.

Gregory M. Kurtzer, cclerget, Michael Bauer, Ian Kaneshiro, David Trudgian, and David Godlove. hpcng/singularity: Singularity 3.7.3, April 2021. https://doi.org/10.5281/zenodo.4667718.

Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. In *The Fourteenth International Conference on Learning Representations*, 2026. https://openreview.net/forum?id=lKEdGCoDNC.

Pat Langley, Herbert A Simon, Gary L Bradshaw, and Jan M Zytkow. Scientific discovery, 1987.

Annan Li, Chufan Wu, Zengle Ge, Yee Hin Chong, Zhinan Hou, Lizhe Cao, Cheng Ju, Jianmin Wu, Huaiming Li, Haobo Zhang, Shenghao Feng, Mo Zhao, Fengzhi Qiu, Rui Yang, Mengmeng Zhang, Wenyi Zhu, Yingying Sun, Quan Sun, Shunhao Yan, Danyu Liu, Dawei Yin, and Dou Shen. The fm agent, 2025. https://arxiv.org/abs/2510.26144.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.

Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025a.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Zexi Liu, Yuzhu Cai, Xinyu Zhu, Yujie Zheng, Runkun Chen, Ying Wen, Yanfeng Wang, Weinan E, and Siheng Chen. Ml-master: Towards ai-for-ai via integration of exploration and reasoning, 2025b. https://arxiv.org/abs/2506.16499.

Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11976–11986, 2022.

Alisia Lupidi, Bhavul Gauri, Thomas Simon Foster, Bassel Al Omari, Despoina Magka, Alberto Pepe, Alexis Audran-Reiss, Muna Aghamelu, Nicolas Baldwin, Lucia Cipolina-Kun, et al. Airs-bench: a suite of tasks for frontier ai research science agents. *arXiv preprint arXiv:2602.06855*, 2026.

Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Sercan O Arik, and Tomas Pfister. MLE-STAR: Machine learning engineering agent via search and targeted refinement. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. https://openreview.net/forum?id=vS1M06Px6u.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

OpenAI. Codex: AI coding agent for ChatGPT. https://chatgpt.com/codex, 2025.

Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-beats: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations*, 2020. https://openreview.net/forum?id=r1ecqn4YwB.

Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for squad. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, 2018.

Tal Ridnik, Emanuel Ben-Baruch, Nadav Zamir, Asaf Noy, Itamar Friedman, Matan Protter, and Lihi Zelnik-Manor. Asymmetric loss for multi-label classification. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 82–91, 2021.

Kristof Schütt, Pieter-Jan Kindermans, Huziel Enoc Sauceda Felix, Stefan Chmiela, Alexandre Tkatchenko, and Klaus-Robert Müller. Schnet: A continuous-filter convolutional neural network for modeling quantum interactions. *Advances in neural information processing systems*, 30, 2017.

Guillem Simeon and Gianni De Fabritiis. Tensornet: Cartesian tensor representations for efficient learning of molecular potentials. *Advances in Neural Information Processing Systems*, 36:37334–37353, 2023.

Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.

Gilbert Syswerda. A study of reproduction in generational and steady-state genetic algorithms. In *Foundations of genetic algorithms*, volume 1, pages 94–101. Elsevier, 1991.

Thesis Labs. Thesis is state-of-the-art on MLE-bench. https://thesislabs.ai/writings/sota-mle-bench, 2025. Accessed: 2026-02-25.

Philipp Thölke and Gianni De Fabritiis. Equivariant transformers for neural network based molecular potentials. In *International Conference on Learning Representations*, 2022. https://openreview.net/forum?id=zNHzqZ9wrRB.

Edan Toledo, Karen Hambardzumyan, Martin Josifoski, RISHI HAZRA, Nicolas Baldwin, Alexis Audran-Reiss, Michael Kuchnik, Despoina Magka, Minqi Jiang, Alisia Maria Lupidi, Andrei Lupu, Roberta Raileanu, Tatiana Shavrina, Kelvin Niu, Jean-Christophe Gagnon-Audet, Michael Shvartsman, Shagun Sodhani, Alexander H Miller, Abhishek Charnalia, Derek Dunfield, Carole-Jean Wu, Pontus Stenetorp, Nicola Cancedda, Jakob Nicolaus Foerster, and Yoram Bachrach. AI research agents for machine learning: Search, exploration, and generalization in MLE-bench. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. https://openreview.net/forum?id=RwfrdKSgCE.

Haorui Wang, Marta Skreta, Cher Tian Ser, Wenhao Gao, Lingkai Kong, Felix Strieth-Kalthoff, Chenru Duan, Yuchen Zhuang, Yue Yu, Yanqiao Zhu, Yuanqi Du, Alan Aspuru-Guzik, Kirill Neklyudov, and Chao Zhang. Efficient evolutionary search over chemical space with large language models. In *The Thirteenth International Conference on Learning Representations*, 2025a. https://openreview.net/forum?id=awWiNvQwf3.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025b. https://openreview.net/forum?id=OJd3ayDDoF.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. https://openreview.net/forum?id=1PL1NIMMrw.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024. https://openreview.net/forum?id=BAakY1hNKS.

Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search, 2025. https://arxiv.org/abs/2504.08066.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

Cunxi Yu, Rongjian Liang, Chia-Tung Ho, and Haoxing Ren. Autonomous code evolution meets np-completeness. *arXiv preprint arXiv:2509.07367*, 2025.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

Ruiyi Zhang, Peijia Qin, Qi Cao, Li Zhang, and Pengtao Xie. Aibuildai: An ai agent that automatically builds ai models, 2026.

Xinyu Zhu, Yuzhu Cai, Zexi Liu, Bingyang Zheng, Cheng Wang, Rui Ye, Jiaao Chen, Hanrui Wang, Wei-Chen Wang, Yuzhi Zhang, et al. Toward ultra-long-horizon agentic science: Cognitive accumulation for machine learning engineering. *arXiv preprint arXiv:2601.10402*, 2026.

Appendix

A Evaluation Failure: A Concrete Example

To illustrate how implementation bugs can silently corrupt the search signal (Section 2.2), we present a real example from an AI agent solving the LMSYS Chatbot Arena competition on MLE-bench. The agent's solution reported a *perfect* cross-validation log-loss of 0.0, which the search process then treated as the best candidate—despite the underlying model being unremarkable.

The bug. The competition requires predicting which of three outcomes occurred (`winner_model_a`, `winner_model_b`, or `winner_tie`). The agent converts multi-hot target columns to single labels via `idxmax(axis=1)`, which returns *column name strings*, not integer indices:

```
target_cols = ["winner_model_a", "winner_model_b", "winner_tie"]
train_labels = train_df[target_cols].idxmax(axis=1).values
# Agent comment: "0, 1, 2" -- actually returns strings:
# ['winner_model_b', 'winner_model_a', 'winner_model_b', ...]
```

When computing the validation metric, the agent passes `labels=[0, 1, 2]` (integers), creating a type mismatch:

```
loss = log_loss(train_labels[val_idx], val_pred, labels=[0, 1, 2])
# Returns 0.0 regardless of predictions
```

Because `scikit-learn`'s `log_loss` cannot match the string ground-truth labels (`'winner_model_a'`, etc.) with the integer label list (`[0, 1, 2]`), it silently returns 0.0 for *any* set of predictions:

```
>>> train_labels[:3]
array(['winner_model_b', 'winner_model_a', 'winner_model_b'], dtype=object)
>>> val_pred = [[0.3, 0.3, 0.4]] * 3  # arbitrary predictions
>>> log_loss(train_labels[:3], val_pred, labels=[0, 1, 2])
0.0
```

Why this matters for search. This bug is particularly insidious in the context of search-based agents. A greedy or evolutionary search process that relies on self-reported validation scores would select this solution as the global optimum, halting further exploration. The solution would persist as the "best" candidate indefinitely, while its actual test performance would be no better than chance. This example concretely demonstrates why decoupling the search signal from agent-controlled evaluation—as in our Hidden Consistent Evaluation protocol (Section 3.3)—is essential for reliable long-horizon search.

B MLE-bench

Table 2 : Per-task Percentile Rank scores for $AIRA_2$ at different time cutoffs. Values shown as score with 95% CI below.

Task	3h	12h	24h	72h
aptos2019-blindness-detection	70.30 51.16, 93.31	87.66 64.79, 99.76	97.31 93.75, 99.76	97.35 96.41, 98.16
billion-word-imputation	66.67 8.05, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
bms-molecular-translation	19.72 19.45, 20.25	28.15 19.45, 34.10	57.67 32.95, 80.66	69.07 60.64, 85.93
cassava-leaf-disease-classification	58.85 39.49, 88.97	87.38 82.90, 90.26	84.73 77.59, 90.26	89.15 86.36, 92.13
champs-scalar-coupling	30.72 9.02, 45.80	49.17 33.02, 64.10	63.12 41.09, 94.34	78.13 60.56, 98.79
freesound-audio-tagging-2019	98.16 97.60, 98.56	99.68 99.04, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
h-and-m-personalized-fashion-recommendations	27.62 27.06, 28.65	29.03 28.75, 29.53	29.21 28.95, 29.53	29.60 29.19, 30.00
hms-harmful-brain-activity-classification	30.30 29.50, 31.09	30.57 29.25, 31.89	30.28 29.25, 31.31	31.76 31.63, 31.89
hotel-id-2021-fgvc8	85.51 82.61, 89.13	89.13 89.13, 89.13	89.86 89.13, 91.30	94.57 92.39, 95.65
hubmap-kidney-segmentation	89.27 87.32, 90.49	93.27 89.49, 97.58	93.91 89.49, 99.50	98.92 97.58, 99.58
imet-2020-fgvc7	38.60 37.89, 40.00	52.63 46.32, 55.79	71.93 47.37, 84.21	85.61 83.16, 88.42
jigsaw-unintended-bias-in-toxicity-classification	8.05 7.86, 8.39	8.05 7.90, 8.36	8.06 7.90, 8.36	8.37 7.94, 8.85
kuzushiji-recognition	86.69 69.97, 97.61	94.43 84.30, 100.00	95.34 86.01, 100.00	97.50 92.49, 100.00
mlsp-2013-birds	96.67 95.00, 98.75	97.92 96.25, 98.75	98.33 97.50, 98.75	98.75 97.50, 100.00
multi-modal-gesture-recognition	62.96 38.89, 100.00	70.37 44.44, 100.00	70.37 44.44, 100.00	81.48 55.56, 100.00
new-york-city-taxi-fare-prediction	37.47 35.58, 40.09	38.79 35.58, 40.50	38.79 35.58, 40.50	37.04 32.95, 40.30
nfl-player-contact-detection	42.49 23.22, 79.23	71.07 25.03, 94.46	71.92 26.30, 94.99	82.73 58.09, 95.63
nomad2018-predict-transparent-conductors	99.66 99.54, 99.89	99.58 99.54, 99.66	99.66 99.54, 99.89	99.77 99.66, 100.00
osic-pulmonary-fibrosis-progression	13.85 12.40, 15.70	10.89 10.07, 11.50	11.02 10.07, 11.88	12.50 11.88, 13.22
petfinder-pawpularity-score	54.37 52.78, 55.75	56.41 54.11, 59.88	63.37 51.12, 83.74	88.62 68.45, 99.72
plant-pathology-2021-fgvc8	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
smartphone-decimeter-2022	4.89 4.89, 4.89	5.00 4.89, 5.06	5.00 4.89, 5.06	9.02 4.89, 16.75
spooky-author-identification	97.29 96.45, 98.15	98.04 97.99, 98.07	97.90 97.26, 98.31	98.39 98.31, 98.47
stanford-covid-vaccine	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
tensorflow2-question-answering	48.91 48.66, 49.15	94.30 88.97, 97.89	97.27 96.51, 97.89	97.73 96.51, 98.54
tweet-sentiment-extraction	73.34 60.39, 83.12	96.28 95.95, 96.49	95.92 93.57, 98.25	93.73 85.86, 98.16
us-patent-phrase-to-phrase-matching	96.52 91.53, 99.15	99.51 99.21, 99.89	99.63 99.42, 100.00	99.91 99.79, 100.00
uw-madison-gi-tract-image-segmentation	18.06 4.91, 43.83	22.54 5.30, 56.43	26.18 5.04, 67.61	33.35 5.04, 89.08
ventilator-pressure-prediction	48.78 47.96, 49.39	55.40 50.02, 63.52	55.93 50.38, 63.52	61.78 60.83, 63.52
whale-categorization-playground	92.11 86.17, 96.59	98.55 97.35, 99.24	98.55 97.35, 99.24	99.31 98.30, 99.81

Table 3 : Per-task test percentile scores for $\text{AIRA}_2^\dagger$ at different time cutoffs. Values shown as score with 95% CI below.

Task	3h	12h	24h	72h
aptos2019-blindness-detection	92.59 83.67, 98.22	95.75 88.70, 99.39	95.75 88.70, 99.39	96.60 93.03, 99.25
billion-word-imputation	97.70 93.10, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
bms-molecular-translation	16.40 0.00, 49.20	80.97 73.68, 85.13	86.08 84.44, 87.41	93.63 92.56, 95.08
cassava-leaf-disease-classification	87.98 82.90, 94.69	88.40 82.90, 95.95	96.60 90.26, 99.95	99.06 97.26, 99.97
champs-scalar-coupling	95.05 92.70, 96.38	99.06 98.69, 99.31	99.27 99.01, 99.45	99.53 99.49, 99.56
freesound-audio-tagging-2019	95.84 93.05, 97.84	99.36 98.08, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
h-and-m-personalized-fashion-recommendations	29.03 28.95, 29.19	30.82 29.22, 31.83	33.53 29.46, 36.40	34.07 29.22, 36.57
hms-harmful-brain-activity-classification	24.21 21.04, 26.32	27.90 24.37, 31.31	28.48 26.68, 30.73	28.21 26.46, 31.49
hotel-id-2021-fgvc8	91.30 90.22, 92.39	95.29 93.48, 96.74	97.10 96.74, 97.83	98.55 96.74, 100.00
hubmap-kidney-segmentation	89.27 85.15, 92.16	95.25 89.41, 98.42	96.19 92.58, 99.67	98.25 97.50, 99.67
imet-2020-fgvc7	64.21 53.68, 83.16	85.26 85.26, 85.26	87.72 85.26, 90.53	92.63 91.58, 94.74
jigsaw-unintended-bias-in-toxicity-classification	7.81 7.75, 7.90	8.20 7.90, 8.70	8.34 7.98, 8.70	8.66 8.36, 9.23
kuzushiji-recognition	99.54 98.63, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
mlsp-2013-birds	95.00 90.00, 98.75	96.67 93.75, 98.75	99.17 98.75, 100.00	99.17 98.75, 100.00
multi-modal-gesture-recognition	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
new-york-city-taxi-fare-prediction	39.71 36.05, 42.45	38.54 36.05, 41.78	36.97 36.05, 37.80	38.63 36.66, 40.90
nfl-player-contact-detection	92.08 88.29, 94.46	95.03 94.14, 95.95	95.56 93.82, 96.81	96.77 96.06, 97.12
nomad2018-predict-transparent-conductors	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
osic-pulmonary-fibrosis-progression	12.56 12.31, 12.88	12.53 11.12, 14.17	12.02 11.12, 13.07	12.34 12.17, 12.45
petfinder-pawpularity-score	75.26 58.75, 99.46	94.19 93.13, 96.21	98.13 96.10, 100.00	98.70 96.10, 100.00
plant-pathology-2021-fgvc8	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
smartphone-decimeter-2022	8.96 4.89, 16.75	40.78 4.89, 100.00	36.77 4.89, 100.00	36.59 4.89, 100.00
spooky-author-identification	98.25 98.15, 98.31	98.74 98.55, 98.95	98.93 98.87, 98.95	99.03 98.95, 99.11
stanford-covid-vaccine	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
tensorflow2-question-answering	48.91 48.66, 49.07	80.35 49.47, 97.40	95.59 92.62, 97.97	98.54 97.89, 99.43
tweet-sentiment-extraction	88.13 79.00, 96.31	97.93 96.90, 98.52	98.53 98.20, 99.10	98.25 97.44, 99.10
us-patent-phrase-to-phrase-matching	99.51 99.42, 99.68	99.93 99.84, 100.00	100.00 100.00, 100.00	100.00 100.00, 100.00
uw-madison-gi-tract-image-segmentation	61.39 57.27, 68.07	81.02 59.21, 93.21	87.90 80.38, 93.21	93.95 89.40, 97.29
ventilator-pressure-prediction	46.17 44.62, 47.20	53.94 52.34, 55.76	58.74 55.57, 61.18	70.06 60.48, 88.10
whale-categorization-playground	94.63 91.86, 96.59	96.84 96.21, 97.35	98.36 97.35, 99.43	99.56 99.43, 99.81

C AIRS-Bench

Table 4: Per-task results on AIRS-Bench. Each task group shows AIRA_2 followed by the two best-performing baselines (selected by best seed). Normalised scores are scaled so that 0 = worst and 100 = SOTA. ∞ denotes normalised scores whereby 1 seed achieved the optimal score. This is usually due to unfair solutions. See Section 4.6. We show the best baseline per task selected by the best seed score since we care about SOTA beating solutions. **Bold** indicates the best method per task (by mean normalised score).

Task	Method	SOTA	Best	Mean	Norm. Best	Norm. Mean	Seeds	Beats SOTA
Code Gen.	**AIRA$_2$**	0.1870	**0.2810**	**0.2309**	**159.4**	**127.3**	**3**	**3**
	$AIRA_{\text{greedy}}$ GPT5		0.0050	0.0020	2.4	1.0	10	0
Code Retrieval	**AIRA$_2$**	0.6113	**0.5772**	**0.5037**	**91.1**	**74.8**	**3**	**0**
	$AIRA_{\text{greedy}}$ GPT5		0.4485	0.1968	63.0	23.2	10	0
Coref. (WSC)	**AIRA$_2$**	0.9620	**1.0000**	**0.9952**	∞	∞	**4**	**4**
	$AIRA_{\text{greedy}}$ GPT5		0.9423	0.7577	85.2	34.2	10	0
Coref. (Wino.)	**AIRA$_2$**	0.8540	**0.9132**	**0.9092**	**130.4**	**127.9**	**4**	**4**
	$AIRA_{\text{greedy}}$ GPT5		0.9148	0.6938	131.5	56.6	10	4
Mol. Prop. (C_v)	**AIRA$_2$**	0.0210	**0.0193**	**0.0199**	**100.8**	**100.5**	**4**	**4**
	$AIRA_{\text{greedy}}$ GPT5		0.0306	0.0557	96.4	90.8	10	0
Mol. Prop. (G)	**AIRA$_2$**	7.5300	**5.5464**	**5.6387**	**101.1**	**101.0**	**4**	**4**
	$AIRA_{\text{greedy}}$ o3		17.6286	3.39×10^4	97.0	70.0	10	0
Graph Reg.	**AIRA$_2$**	0.0170	**0.0226**	**0.0227**	**93.9**	**93.7**	**4**	**0**
	$AIRA_{\text{greedy}}$ o3		0.0408	0.0984	81.1	62.0	10	0
Math QA	**AIRA$_2$**	0.9420	**0.9367**	**0.9267**	**96.9**	**92.2**	**4**	**0**
	$AIRA_{\text{greedy}}$ o3		0.6500	0.4253	36.9	19.5	10	0
QA (DuoRC)	**AIRA$_2$**	0.4648	**0.3925**	**0.3842**	**79.7**	**77.6**	**4**	**0**
	$AIRA_{\text{oneshot}}$ OSS-120B		0.3262	0.2782	63.2	52.2	20	0
QA (ELI5)	**AIRA$_2$**	0.2690	**0.2523**	**0.2483**	**92.7**	**91.0**	**4**	**0**
	$AIRA_{\text{greedy}}$ GPT5		0.2354	0.1881	85.6	66.2	10	0
QA (FinQA)	**AIRA$_2$**	0.7803	**1.0000**	**0.6713**	∞	∞	**4**	**1**
	$AIRA_{\text{greedy}}$ GPT5		0.0924	0.0340	6.4	2.3	10	0
Mol. Prop. (R^2)	**AIRA$_2$**	0.0330	**0.0221**	**0.0241**	**103.6**	**102.8**	**4**	**4**
	$AIRA_{\text{greedy}}$ GPT5		0.1550	0.5619	86.2	74.7	10	0
Reading Comp.	**AIRA$_2$**	0.8580	**0.8437**	**0.8316**	**95.1**	**91.4**	**4**	**0**
	$AIRA_{\text{greedy}}$ GPT5		0.8645	0.5187	102.4	37.5	10	2
Sentiment	**AIRA$_2$**	0.7780	**0.7311**	**0.7304**	**85.1**	**84.9**	**4**	**0**
	$AIRA_{\text{greedy}}$ GPT5		0.7165	0.6671	80.9	68.4	10	0
Text Cls.	**AIRA$_2$**	0.9050	**0.9333**	**0.9298**	**116.1**	**113.8**	**4**	**4**
	$AIRA_{\text{greedy}}$ o3		0.9327	0.9012	115.7	98.2	10	9
Text Sim.	**AIRA$_2$**	0.8540	**0.9083**	**0.9070**	**124.1**	**123.3**	**4**	**4**
	$AIRA_{\text{greedy}}$ GPT5		0.9078	0.8737	123.8	107.5	10	8
TS (Traffic)	**AIRA$_2$**	0.6220	4.72×10^{11}	4.72×10^{11}	**18.8**	**12.5**	**3**	**0**
	$AIRA_{\text{greedy}}$ OSS-120B		4.28×10^{11}	5.82×10^{12}	19.1	11.4	10	0
TS (Rideshare)	**AIRA$_2$**	1.1850	**1.0661**	**1.1528**	**105.6**	**101.5**	**3**	**1**
	$AIRA_{\text{greedy}}$ OSS-20B		1.1741	1.2884	100.5	95.6	10	2
TS (Solar)	**AIRA$_2$**	576.35	8.06×10^2	1.06×10^3	**83.7**	**72.0**	**3**	**0**
	$AIRA_{\text{greedy}}$ GPT5		7.49×10^2	1.29×10^3	87.3	61.1	10	0
Mol. Prop. (U_0)	**AIRA$_2$**	5.83	**4.4481**	**6.8336**	**101.0**	**99.9**	**4**	**3**
	$AIRA_{\text{greedy}}$ GPT5		11.9779	77.3832	97.5	90.9	10	0

D Compute Optimal Resource Allocation

Since $P = 100 \cdot g/(g+1)$ is strictly monotonically increasing in g, and $g = \alpha \cdot h$ with $\alpha > 0$, maximizing P over (N, t) subject to a fixed compute budget $C = N \cdot t$ reduces to maximizing

$$h(N, t) = \log(\gamma\, t + 1) \cdot \log(\beta\, N + 1). \tag{6}$$

Theorem 1. *Given the compute budget constraint $C = N \cdot t$ with $C, \gamma, \beta > 0$, the function $h(N, t) = \log(\gamma\, t + 1) \cdot \log(\beta\, N + 1)$ achieves its unique global maximum at:*

$$N^* = \sqrt{\frac{\gamma C}{\beta}}, \qquad t^* = \sqrt{\frac{\beta C}{\gamma}}. \tag{7}$$

Proof. Substituting $t = \frac{C}{N}$ into the objective, we obtain a single-variable function $h(N) = \log\left(\frac{\gamma C}{N} + 1\right) \cdot \log(\beta\, N + 1)$. To find the stationary points, we introduce auxiliary variables $x \triangleq \beta N$ and $y \triangleq \frac{\gamma C}{N}$. Note that $x \cdot y = \beta\gamma C$, which is a constant independent of N.

Setting the derivative of $h(N)$ with respect to N to zero yields:

$$\frac{dh}{dN} = \frac{\beta}{1 + \beta N} \log\left(1 + \frac{\gamma C}{N}\right) - \frac{\gamma C / N^2}{1 + \frac{\gamma C}{N}} \log(1 + \beta N) = 0. \tag{8}$$

Substituting x, y, and the identities $\beta = \frac{x}{N}$ and $\frac{\gamma C}{N^2} = \frac{y}{N}$ into the stationarity condition, we have:

$$\frac{x/N}{1 + x} \log(1 + y) - \frac{y/N}{1 + y} \log(1 + x) = 0. \tag{9}$$

Multiplying by N and separating the variables to opposite sides of the equation gives:

$$\frac{1 + x}{x} \log(1 + x) = \frac{1 + y}{y} \log(1 + y). \tag{10}$$

Observe that this equation is of the form $f(x) = f(y)$, where $f(z) \triangleq \frac{1+z}{z} \log(1 + z)$. To determine the injectivity of $f(z)$ for $z > 0$, we evaluate its derivative:

$$f'(z) = \frac{z - \log(1 + z)}{z^2}. \tag{11}$$

By the standard logarithmic inequality $z > \log(1 + z)$ for all $z > 0$, it follows that $f'(z) > 0$. Therefore, $f(z)$ is strictly monotonically increasing, implying that $f(x) = f(y)$ if and only if $x = y$.

Equating $x = y$ and substituting their original definitions back yields:

$$\begin{aligned} \beta N &= \frac{\gamma C}{N} \\ N^* &= \sqrt{\frac{\gamma C}{\beta}}. \end{aligned} \tag{12}$$

Applying the constraint $t = \frac{C}{N}$ gives the corresponding optimal time allocation $t^* = \sqrt{\frac{\beta C}{\gamma}}$, concluding the proof. □

Remark (Optimal Performance Trajectory). At the optimum, both logarithmic arguments equalize to $\sqrt{\beta\gamma C} + 1$, so $h^* = \left[\log\left(\sqrt{\beta\gamma C} + 1\right)\right]^2$ and $g^* = \alpha \cdot h^*$. Substituting into $P = 100 \cdot g/(g+1)$ gives the maximum achievable performance as a function of the total compute budget C:

$$P(C) = 100 \cdot \frac{\alpha \left[\log\left(\sqrt{\beta\gamma C} + 1\right)\right]^2}{\alpha \left[\log\left(\sqrt{\beta\gamma C} + 1\right)\right]^2 + 1}. \tag{13}$$

Remark (Integer Constraint). Since N must be a positive integer in practice, we round N^* to the nearest integer, $N^* \leftarrow [N^*]$, and set $t^* = C/N^*$.